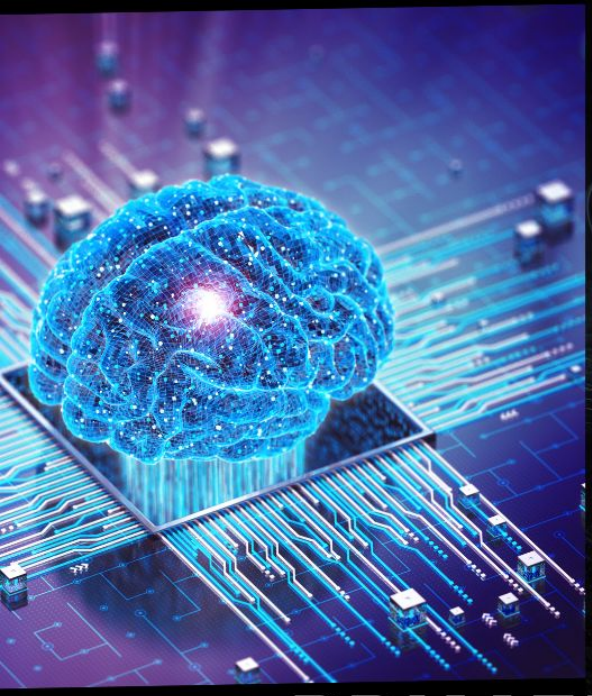
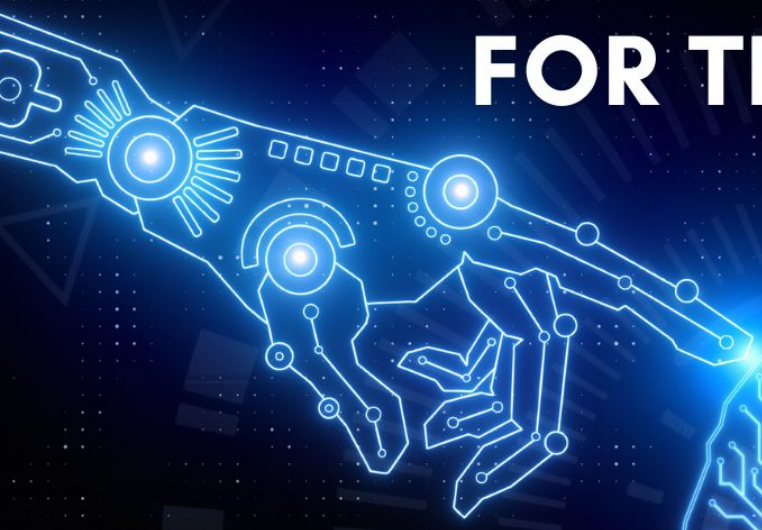




PRIMER ON ROBOTICS AND ARTIFICIAL INTELLIGENCE FOR TEACHERS

CLASSES IX & X







COUNCIL FOR THE INDIAN SCHOOL CERTIFICATE EXAMINATIONS

Primer on Robotics and Artificial Intelligence for Teachers

Classes IX & X

Research Development and Consultancy Division (RDCD)

Council for the Indian School Certificate Examinations
New Delhi

Edition: August 2024

© Copyright, Council for the Indian School Certificate Examinations

All rights reserved. The copyright to this publication and any part thereof solely vests in the Council for the Indian School Certificate Examinations. This publication and no part thereof may be reproduced, transmitted, distributed or stored in any manner whatsoever, without the prior written approval of the Council for the Indian School Certificate Examinations.

Council for the Indian School Certificate Examinations (CISCE)

MISSION STATEMENT

The Council for the Indian School Certificate Examinations is committed to serving the nation's children, through high quality educational endeavours, empowering them to contribute towards a humane, just and pluralistic society, promoting introspective living, by creating exciting learning opportunities, with a commitment to excellence.

ETHOS OF CISCE

- Trust and fair play.
- Minimum monitoring.
- Allowing schools to evolve their own niche.
- Catering to the needs of the children.
- Giving freedom to experiment with new ideas and practices.
- Diversity and plurality - the basic strength for evolution of ideas.
- Schools to motivate pupils towards the cultivation of:
  Excellence - The Indian and Global experience.
  Values - Spiritual and cultural - to be the bedrock of the educational experience.
- Schools to have an 'Indian Ethos', strong roots in the national psyche and be sensitive to national aspirations.

Preface

Robotics and Artificial Intelligence (AI) play crucial roles in modern society and have a significant impact on a plethora of industrial and human activities enhancing precision and efficiency, bringing advancements in healthcare, education, governance, finance and national security as well as scientific explorations and research. The National Education Policy 2020 also lays emphasis on the development of basic digital skills in all students, to build readiness towards the emerging future careers and thrive in this digital age. CISCE recognises the transformative role of Robotics and AI in all professions and vocations and thus has introduced the subject at the ICSE Level for the Examination Year 2025. This subject offers the opportunity to foster knowledge and skills in secondary level students on the foundational theory and practical aspects of Robotics and AI.

The objective of this subject primer is to delineate the specific goals for this subject on Robotics and Artificial Intelligence (AI) at the ICSE Level. This primer provides a framework for the orderly development of the basic conceptual knowledge of Robotics and AI in students through experiential learning strategies. It discusses the scope of the CISCE syllabus, along with relevant activities based on the needs and interests of students as well as sample lesson plans and assignments. Along with topic wise activities, certain engaging ideas for the Robotics & AI Clubs in schools are also provided.

I acknowledge the contributions of the content curators and designers who have worked diligently to develop this resource. I hope teachers and students would benefit from the brief and lucid flow of the document with a clarity on the objectives, learning outcomes, pedagogies and scope of the subject Robotics and AI at the ICSE Level.

This is the first edition of the primer, which will be periodically updated to reflect future syllabus revisions and emerging trends in the fields of Robotics and Artificial Intelligence (AI).

August 2024

Dr. Joseph Emmanuel
Chief Executive & Secretary

Contents

I. Introduction to CISCE Robotics and AI Curriculum at the ICSE level 1

II. Relevance of the subject 4

III. Syllabus Class IX

Robotics - Overview of Syllabus 6

1. Introduction to Robotics 7
2. Robot as a System 11
3. Concepts in Robotics 14

Artificial Intelligence – Overview of Syllabus 16

1. Introduction to Artificial Intelligence 18
2. Role of Data and Information, Evolution of Computing 22
3. Introduction to Data and Programming with Python 25
4. AI Concepts and AI Project Framework 32

IV. Syllabus Class X

Robotics - Overview of Syllabus 37

1. New Age Robotics Systems (NARS) Overview 39
2. From Robots to Cobots 40
3. Components of Robots as a System 42
4. Visualization, Design and Creation of Components 44
5. Integrating Robots as a System 45

Artificial Intelligence – Overview of Syllabus 47

1. Decision-Making in Machines 49
2. Machine Intelligence and Cybersecurity in Computing 50
3. Components of AI Project Framework 51
4. Introduction to Data and Programming with Python 54

V. Suggestive Interactive Activities and Assignments for Classes IX & X 58

VI. Some General Activities for Robotics and AI Club 86

VII. Sample Lesson Plan 89

VIII. Future Pathways in Robotics and AI 93

IX. Inter-disciplinary connections of Robotics and AI 95



I. Introduction to CISCE Robotics and AI Curriculum at the ICSE level

This curriculum introduces students to the dynamic realms of Robotics and Artificial Intelligence (AI) with the goals of fostering a deep understanding, developing practical competencies, and instilling essential AI-Readiness skills. Through a combination of classroom instruction, hands-on activities, and project-based learning, students gain insights into key AI concepts, including Data, Computer Vision, and Natural Language Processing. The curriculum also emphasizes the significance of computational skills, particularly in basic Python coding, and raises awareness about the ethical considerations inherent in AI. Overall, the curriculum aims to equip students for the transformative and responsible exploration of these cutting-edge technologies.

A. Curriculum Objectives:

- Define and explain the fundamental features of robots.
- Investigate the historical evolution of robots.
- Engage in discussions on ethical considerations using Isaac Asimov's Three Laws of Robotics.
- Categorize robots based on their workplace and control methods to achieve a comprehensive understanding.
- Demonstrate real-world applications, delve into the study of robots as systems, and enhance learning through hands-on activities, staying updated on the latest applications, and distinguishing between traditional and collaborative robots.
- Acquire knowledge of AI concepts and history, emphasizing the significance of the Turing Test.
- Understand how AI positively impacts various domains and its practical applications.
- Develop a foundational understanding of ethical considerations in AI, highlighting fairness, accountability, and transparency.
- Recognize the importance of data in AI and differentiate between deterministic and probabilistic problem-solving.
- Illustrate the limitations of deterministic computing in real-life problem-solving and emphasize the shift to probabilistic computing in AI.

B. Learning Outcomes:

Students are expected to:

- Possess a robust grasp of fundamental concepts in robotics and understand the historical evolution and ethical considerations in the field of robotics.
- Categorize robots proficiently based on workplace and control methods and gain practical insights into real-world applications of robotics.
- Comprehend robots as systems, analyzing and designing them based on motion, joints, and links, and identify various types of robots and their functionalities.
- Develop a holistic understanding of contemporary robotic applications.
- Acquire knowledge of continuous sensing and familiarity with integral components.
- Gain practical skills in visualizing, designing, and progressing to complex structures in robotic systems.
- Develop a comprehensive understanding of Artificial Intelligence, encompassing its concept, historical development, and the significance of the Turing Test, with the ability to articulate diverse applications and benefits.



- Cultivate an appreciation for the pivotal role of data in AI, recognizing its importance in both deterministic and probabilistic problem-solving, while understanding the limitations of deterministic computing.
- Attain practical proficiency in Python programming, covering various data types, operators, and programming constructs relevant to AI development.
- Establish a foundational grasp of Machine Learning, recognizing its role in subjective decision-making and practical applications such as fruit sorting.
- Differentiate between human and machine decision-making, comprehend the impact of data and algorithms on machine decision processes, and develop awareness of cybersecurity principles and ethical considerations.
- Familiarize with the AI Project Framework, gaining the ability to identify problems, acquire and explore data, and understand data modeling and evaluation within AI projects.

C. Pedagogy: Future Focused Learning and Skilling Approach

The subject will adopt a learner-centric pedagogy to deliver the content. The instructor should adopt a balance between teaching and facilitation methods to enable learning and competence building in the students in the Robotics and AI Subject.

The instructor should engage with the learner using a mix of teaching learning methods which involve interactive lectures, hands on exercises and project-based learning methods. An illustration of applying the teaching learning methods is presented in the following section. For effective delivery of Robotics and AI subject, instructor should include digital tools including Tinkercad and open-source platforms such as Arduino and Arduino IDE and Python programming language as part of the hands-on teaching-learning process.

(i) Interactive Lectures:

The subject Robotics and AI course begins with dynamic lectures, incorporating YouTube videos, debates, movies, and recent news articles to introduce concepts and applications. Visual aids, real-life examples, and case studies enhance engagement, while student participation is encouraged through discussions. The assessment is diverse, encompassing quizzes, assignments, projects, and exams to gauge comprehension and application of the material, creating an interactive and comprehensive learning experience.

Example: Group Discussion on **"Robotics and AI Collaboration for Future Advancements"**

Description: Discuss the symbiotic relationship between Robotics and AI, exploring how their integration is shaping diverse industries. Delve into the potential benefits, challenges, and ethical considerations, emphasizing the innovative applications that arise from this integration.

(ii) Hands-on Exercises:

Engage in practical activities with diverse sensors and actuators, mastering programming through hands-on exercises using Arduino and Python. Receive step-by-step guidance and progressively challenging coding tasks to enhance language proficiency and build advanced programming skills.

Example: Design and implement a basic obstacle detection system using an IR sensor and microcontroller.

Example**:** Guess the Number Game: Create a simple AI program that guesses the number the user is thinking of by implementing a basic search algorithm.



(iii) Project-Based Learning:

Project-based learning plays a vital role in teaching robotics and AI by allowing students to apply theoretical knowledge to real-world applications. Collaborative group projects improve teamwork, problem-solving, and communication skills, mirroring real-world AI and robotics integration. Incorporating industry case studies reinforces theoretical learning and highlights the impact of AI and robotics in fields like healthcare and finance.

Students can make projects on

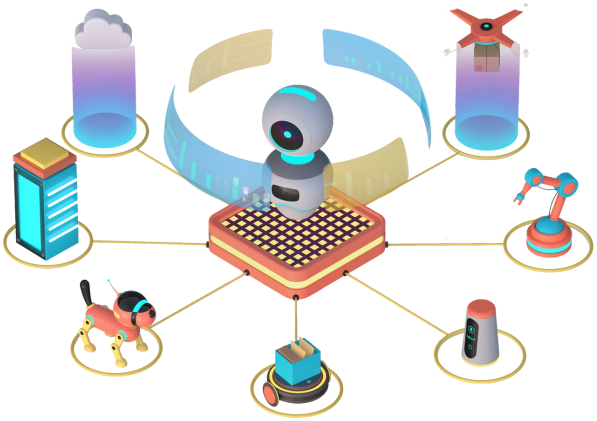
- Voice-Activated Virtual Assistant: Build a voice-controlled virtual assistant that can perform tasks such as setting reminders, answering questions, or providing information.
- Health Monitoring App: Develop an application that uses AI to monitor and analyze health-related data, providing insights into a person's well-being.
- Remote-Controlled Robot: Design and construct a remote-controlled robot equipped with motorized wheels, wireless communication, and obstacle detection sensors for fluid and responsive navigation.
- Board Erasing Robot: Create an autonomous board-erasing robot featuring infrared sensors or computer vision for boundary detection, motorized erasing mechanisms, and efficient surface coverage.

(iv) Ethical Discussions:

- Conduct discussions on ethical considerations in AI and Robotics, addressing bias, fairness, and transparency.
- Encourage critical thinking about ethical implications.
- Multimedia Resources: Utilize multimedia like videos, animations, and simulations to enhance understanding and engagement.

(v) Guest Speakers and Study Trips:

- Arrange guest lectures by AI and Robotics experts.
- Organize field trips to AI and Robotics-related organizations for real-world exposure.



II. Relevance of the subject

Robotics and AI play pivotal roles in our tech-driven world. Robotics automates industries and healthcare, boosting efficiency and safety. Meanwhile, AI empowers machines to learn and revolutionizes data analysis and personalization. Together, they drive innovation in self-driving cars, medical diagnostics, and tackle societal challenges. Embracing these subjects equips individuals for the evolving job market, offering skills to navigate and leverage cutting-edge technologies in our AI-centric world.

A. Alignment with National Education Policy (NEP) 2020:

- **Alignment with NEP 2020**: Both the Robotics and AI course and primer for Classes 9th and 10th adhere to NEP 2020's emphasis on a multidisciplinary approach and technology integration.
- **Fostering Key Skills and Competencies:** The curriculum promotes experiential learning, critical thinking, problem-solving skills, skill development, and ethical considerations, aligning with NEP 2020's goals for holistic, value-based, and practical education.
- **Pedagogical Approach**: Utilizes a blend of traditional and innovative teaching methods, supporting NEP 2020's focus on flexibility and student-centered learning.

B. Alignment with National Curriculum Framework for School Education (NCF-SE) 2023:

- Child-Centric Approach: The course focuses on engaging and experiential learning, tailored to children's interests and needs.
- Ethical and Bio-Inspired Learning: It includes exploration of ethical issues and bio-inspired designs, ensuring a contextually relevant curriculum.
- Multidisciplinary Robotics: The course covers categorizing robots and their diverse applications, fostering multidisciplinary understanding.

C. Mapping to National Skill Qualification Framework (NSQF) Standards:

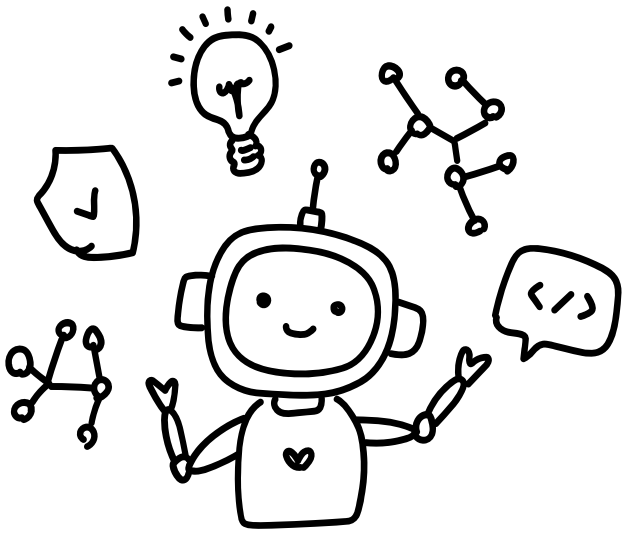
- The course aligns with NSQF Standards, contributing to specific skills in robotics through hands-on activities and coding exercises.
- Continuous assessment methods align with NSQF's focus on real-world skill evaluation.
- Exploration of robotic systems aligns with NSQF's tiered levels of skills and knowledge.
- The AI primer's content aligns with NSQF standards, corresponding to secondary education levels.
- Emphasis on skill development, practical application of AI concepts, and addressing ethical considerations align with NSQF's focus on developing essential skills and values.

Note:

Both the Introduction to Robotics and AI Curriculum and the primer effectively align with NEP 2020, emphasizing multidisciplinary learning, skill development, and experiential education. They seamlessly integrate with NCF SE principles, fostering a child-centric approach, ethical considerations, and multidisciplinary understanding. Additionally, their alignment with NSQF Standards ensures a focus on skill development, continuous assessment, and practical application, contributing to a holistic and relevant education.



III. SYLLABUS CLASS IX



Robotics

Overview of Syllabus

1. Introduction to Robotics

(i) Understanding Robots.

Basic understanding of what a robot is; definition and characteristics; benefits of using robots (with respect to humans): increased quality, increased productivity, increased efficiency, longer working span, working in hazardous environments, improved workplace; Limitations.

(ii) Evolution of Robots; Laws of Robotics.

Brief history of Robots with respect to their evolution from 1900's till date. Definition of Robotics, the three Laws of Robotics by Isaac Asimov (statements only).

(iii) Classification of Robots.

Classification of Robots as: field/terrain based (arial, ground, underwater) and control based (manual, automatic): Meaning and examples of each. Bio-inspired robots: meaning, purpose and examples (humanoids, birds, snakes and insects).

(iv) Real world Robots and their applications.

Application of robots in different fields (domestic, industry, medical, defense, entertainment and agriculture) with at least one example of each.

2. Robot as a System

(i) Building blocks of Robots.

General block diagram of a robot. A detailed study of the building blocks of a robot. Concept of a robot as having mechanical, electronic and computational blocks; functioning and working principle of each block. Design aspects using examples of humanoid, aerial, underwater and mobile robots.

(ii) Identification of Robots.

Identification of robots (through demonstration/ video/graphic details). Illustration using an industrial robot (e.g., Industrial Robotic Arm), humanoid and mobile robot. The idea that a mechanical body can be of any form must be emphasized.

3. Concepts in Robotics

(i) Types of motion; motion in one-dimension and two-dimension; types of joints and links.

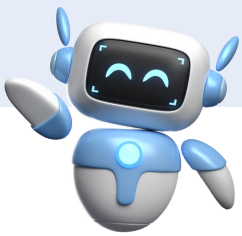
Types of motion (linear, angular, and circular); a brief understanding of motion in one-dimension and two-dimension; types of joints (prismatic, revolute, and spherical); types of links (rigid and soft). Relevant examples for each of the above.

(ii) Using links and joints to create specific motion.

A detailed study of how links and joints help create specific motion. Identification of links and joints used in a given system. Examples for the demonstration can include Industrial Robot Arms.

(iii) Degree of freedom of a robot

Definition; identification through illustration.





1. Introduction to Robotics

1.1. Understanding Robots

Robots are programmable machines that can be designed to carry out tasks automatically or semi-automatically. They can be as simple as industrial robotic arms or as complex as humanoid robots with advanced artificial intelligence. Robots can be used for a variety of tasks, including manufacturing, healthcare, exploration, and entertainment.

1.1.1. Definition and characteristics:

The definition of a robot can vary, but they typically share certain characteristics such as the ability to perceive their environment, process data, and take actions based on that data. They can also be equipped with various sensors, actuators, and computing resources to perform their tasks.

1.1.2. Benefits of using robots:

Robots offer numerous advantages over humans in various applications. They can deliver increased quality and consistency in tasks that require precision. Robots can also increase productivity by working faster and more efficiently than humans. Their ability to work in hazardous environments or perform tasks that are dangerous or physically demanding for humans is another significant benefit. Additionally, robots have a longer working span as they don't get tired and can operate 24/7. The introduction of robots in the workplace can lead to safer work environments and reduced labor costs. However, it's also important to consider the social and economic implications, such as the potential displacement of human workers.

1.1.3. Limitations of using robots:

Some limitations of using robots include:

(a) *Affordability*: Industrial robots require a large upfront investment, including additional installation and configuration costs. Manufacturers also need to consider future maintenance costs and the need for extra components.

(b) *Complexity of Tasks*: Nowadays, AI has improved the robotics capabilities. But, they still struggle with tasks that humans perform effortlessly, especially in unstructured environments.

(c) *Limited Versatility*: Robots often lack the versatility and adaptability of humans in handling a wide range of tasks and situations. Reprogramming and re-configuring them for new tasks may require significant time and effort.

(d) *Environmental Impact*: The production and operation of robots can have environmental consequences, including energy consumption, resource extraction, and waste generation. While robots may offer efficiency gains in certain applications, their environmental footprint should be considered in the broader context of sustainability and environmental stewardship.

(e) *Limited Creativity and Intuition*: Robots lack the creativity, intuition, and emotional intelligence of humans, which can be essential for tasks requiring innovation, empathy, and social interaction. While robots excel at performing repetitive or algorithmic tasks, they may struggle with tasks that require abstract thinking or emotional understanding.

(f) *Safety Concerns*: Safety is a critical consideration when deploying robots, especially in shared workspaces with humans. Robots must be designed with safety features to prevent accidents and injuries, such as collision detection, emergency stop mechanisms, and protective barriers. However, ensuring complete safety in all scenarios can be challenging.



1.2. Evolution of Robots; Laws of Robotics

1.2.1. Laws of Robotics:

In 1941 and 1942, Isaac Asimov formulated the Three Laws of Robotics, and in the process coined the word "robotics".

1. A robot may not injure a human being, or, through inaction, allow a human being to come to harm.
2. A robot must obey the orders given to it by human beings unless that is in conflict with the first law.
3. A robot must protect its own existence as long as such protection does not conflict with the First or Second Law.

Understanding the history of robots can provide valuable context for teachers and students. Here's a brief overview:

1.2.2. First Generation: Manipulator Robots (1950s-1960s):

The inaugural era of robotics witnessed the birth of manipulator robots, prevalent in the latter half of the 20th century. These machines were characterized by simplicity, executing predetermined tasks manually or in a fixed sequence. Classified as open-loop systems, their actions lacked environmental awareness, limiting their movements to specific, repetitive functions. Despite their basic nature, first-generation robots persist in numerous industries, performing tasks such as pick-and-place operations.



Figure 1: The Unimate (1956).

1.2.3. Second Generation: Learning Robots (1980s):



Figure 2: PUMA (Programmable Universal Machine for Assembly).

The 1980s ushered in the second generation, marked by learning robots capable of memorizing sequences of movements performed by human operators. These closed-loop systems exhibited improved mobility and environmental perception through specialized sensors. Widely employed in the automotive industry, they excelled in tasks like painting, transferring, and assembling parts on assembly lines, demonstrating a fundamental shift toward adaptability and basic feedback mechanisms.

1.2.4. Third Generation: Sensorized Control Robots (1990s):



Figure 3: ABB's IRB 6000.

The third generation introduced sensorized control robots, distinguished by their reprogrammability via computers. Equipped with artificial vision and touch capabilities through sensors and programming languages, these robots displayed a greater understanding of their environment. The closed-loop control system allowed for dynamic modifications in work strategies and displacement, marking the emergence of intelligent robots and laying the foundation for the era of adaptable automation.

1.2.5. Fourth Generation: Intelligent Robots (Late 20th Century - Early 21st Century):

Intelligence took center stage in the fourth generation, characterized by robots with real-time decision-making abilities and enhanced freedom of movement. Advancements in fuzzy logic and neural networks facilitated the processing of information gathered by increasingly accurate sensors. These robots found application across diverse sectors, transitioning from industrial applications to services and entertainment. The humanoid appearance of some fourth-generation robots hinted at the merging of technology with anthropomorphic design.



Figure 4: ASIMO Humanoid robot.

1.2.6. Fifth Generation: Advanced Artificial Intelligence (21st Century - Present):

The current pinnacle, the fifth generation, showcases robots utilizing advanced artificial intelligence with autonomous learning capabilities. Breaking free from the notion of being confined to repetitive tasks, these robots can imitate, develop models of performance, and exhibit reasoning and behavior in dynamic and unknown environments. This generation, still in active development, promises limitless applications, reflecting the multifaceted adaptability that sets them apart. The responsibility of further refining and advancing this generation rests with future innovators and technologists.



Figure 5: Tele Operation and Tele Observance (TOTO) Mobile Robot developed by SeiAnmai Technologies Pvt. Limited, incubated at IHFC-IIT Delhi, India

1.2.7. AI and Machine Learning:

Today, robots are integral to industries like manufacturing, healthcare, and logistics. They also play a role in research, education, and even entertainment, with robots like Boston Dynamics' Spot and the use of drones.

Artificial intelligence and machine learning technologies have played a crucial role in the evolution of robots.

Modern robots can adapt to changing environments, learn from their experiences, and make decisions autonomously.

1.3. Classification of Robots

1.3.1. Field/Terrain based classification:

Robots can be classified based on the environments in which they operate:

Arial robots: Designed to fly and operate in the air. Examples include drones and quadcopters.

Ground robots: Operate on solid surfaces. Examples include wheeled robots, tracked robots, and legged robots.

Underwater robots: Designed to operate in aquatic environments. Examples include remotely operated vehicles (ROVs) and autonomous underwater vehicles (AUVs).

1.3.2. Control based classification:

Robots can also be classified based on how they are controlled:

Manual robots: Operated directly by a human operator, often using a remote control or joystick.

Automatic robots: Operate autonomously without human intervention, using sensors and algorithms to make decisions and perform tasks.

Bio-inspired robots: Robots that are designed based on biological organisms and their behaviors. Examples include:

(a) Humanoids: Robots designed to resemble and mimic human behavior and physical appearance. Example: Boston Dynamics' Atlas robot.
(b) Birds: Robots inspired by avian creatures, often designed for aerial applications. Example: Robo Raven, a robot designed to mimic the flight of a bird.
(c) Snakes: Robots designed to mimic the movement of snakes, often used for tasks that require a high degree of flexibility and maneuverability. Example: CMU's Biorobotics Lab's snake robots.
(d) Insects: Robots inspired by insects, often small and agile. Example: RoboBees, a swarm of robot insects designed for pollination.

1.4. Real World Robots and their Applications

1.4.1. Application of robots in different fields:

(a) Domestic: Cleaning robots, personal assistance, smart home integration. Example: Mi Xiaomi Robot Vacuum Cleaner 2Pro: Robotic Vacuums



figure 6: Vacuum Cleaner

(b) Industry: Manufacturing automation, quality control, material handling. Example: TATA Motors automates vehicle painting using robotic arm.



Figure 7: KUKA Robotic Arm.

(c) Medical: Surgical assistance, rehabilitation, telepresence. Example: Manipal Hospital Whitefield Delhi is advancing Robotic Surgery.
(d) Defence: Surveillance drones, bomb disposal robots, autonomous vehicles. Example: Indian Army introduces the MULE: The AI-based robot system that redefines modern warfare.
(e) Entertainment: Theme Park animatronics, media production, educational entertainment. Example: BotLab Robots entertaining innovations

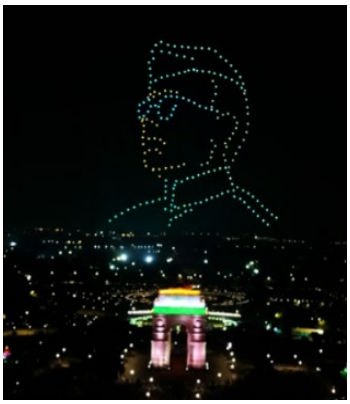


Figure 8: Drone Show by Botlab Dynamic

(f) Agriculture: Precision farming, harvesting, weeding. Example: advanced agriculture spraying drone -DJI - Agras T16

2. Robot as a System

2.1. General block diagram of a Robot

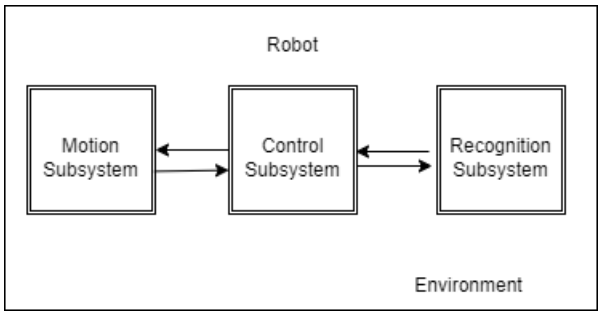


Figure 9: Building blocks of robot.

A typical robot can be represented by a block diagram consisting of the following building blocks (refer Figure 9):

Motion Subsystem: The motion subsystem is the physical structure of the robot that carries out desired motion like human arms.

Recognition Subsystem: The recognition subsystem uses various sensors to gather information about the robot itself and any object being acted upon and about the environment. Based on sensor data, it recognizes the robots state, the objects and the environment.

Control Subsystems: The control subsystem is a device or a set of devices to manage, command, direct or regulate the behavior of the process. The control subsystem influences the robots motion to achieve a given task using the information provided by the recognition subsystem.

2.1.1. Concept of a robot

Robots come alive through a complex interplay of various components such as mechanical, electronic and computational. The fundamental concepts of these components are explained ahead:

2.1.1.1. Mechanical Component

The mechanical components include the following:

Skeletal Structure forms the robot's base, providing support and housing other components. It can be made of metal, plastic, or composite material depending on the robot's needs.

Actuators are the "muscles" that translate electrical signals into movement. Different types of actuators, like servo motors, hydraulics, or pneumatic, are chosen based on the required force, speed, and precision.

Transmission (not always needed) helps transfer power from the actuators to the end effector efficiently, especially in robots with high torque requirements.

End effectors are the parts that directly interact with the environment, like grippers in industrial robots or wheels in mobile robots. Their design varies based on the application and robot's functionality.

Examples:

Bionic Hands are used for gripping of objects. The prosthetic hand fingers mimic that of a human finger.

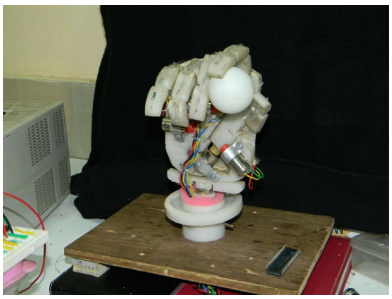


Figure 10: Bionic Hand developed by Tezpur University, Assam, India

2.1.1.2. Electrical and Electronic Component

The essential components involved in the electrical and electronic concepts of a robot are discussed below.

Sensors gather information about the robot's surroundings. Sensors are devices that are electrically powered and designed to detect and measure specific physical properties or changes in the environment. They play a crucial role in a wide range of applications by providing essential data for monitoring, control, and automation.

For Example, Cameras provide vision, Light detection and ranging (LiDAR) offers mapping of the surroundings useful in navigation, and several other sensors to detect touch, force, or proximity.

Data Acquisition System (DAS) collects and processes sensor data, converting it into a format the controller can understand.

Controller is the robot's "brain," interpreting sensor data and sending control signals to actuators. These signals or commands are based on the instructions or algorithms written using several programming languages. It can be a simple microcontroller or a powerful computer depending on the application and robot's complexity.

Power module consists of a battery source and voltage regulation circuits. Most of robotic applications require DC power source for sensors, actuators and controllers.

Examples:

Obstacle avoiding robot

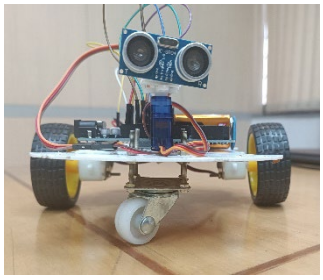


Figure 11: Obstacle-avoiding robot.

2.1.1.3. Computational Component

The essential components involved in the computational concept of a robot.

Software contains algorithms that govern the robot's behaviour, including how it interprets sensor data, makes decisions, and controls actuators. Advanced robots might use machine learning for adaptation and learning.

Controller executes the software, sending signals to actuators based on its calculations. The criteria to select a controller also depends on the computational effort of the software written.

Examples:

Vyommitra is a female-looking humanoid robot developed by the Indian Space Research Organisation to function onboard the Gaganyaan, a crewed orbital spacecraft. Vyommitra was first unveiled on 22 January 2020 at the Human Spaceflight and Exploration Symposium in Bengaluru.

It will accompany Indian astronauts in space missions and be a part of uncrewed experimental Gaganyaan missions before the crewed spaceflight missions.

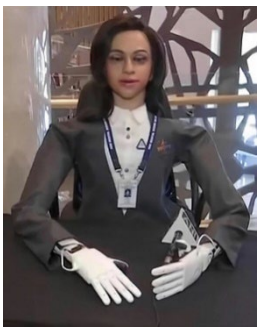


Figure 12: Vyommitra Humanoid robot.

2.2. Identification of Robots

Robots can be identified by their physical appearance, structure, and functionality. They can come in various forms and sizes, from small insect-like robots to large humanoid robots. it's important to emphasize that the mechanical body of a robot can take any form, from a simple wheeled robot to a complex humanoid. The design of the mechanical body

depends on the robot's intended function and the environment in which it will operate.

2.2.1. Industrial Robots:

These robots are commonly used in manufacturing and assembly lines. They are designed to perform repetitive tasks with precision and speed, such as welding, painting, or assembly.



Figure 13: Robotic arm.

2.2.2. Humanoid Robots:

Humanoid robots are designed to resemble the human form to some extent. They are often used in research, entertainment, and assistance applications, such as healthcare and customer service.

2.2.3. Drones (Unmanned Aerial Vehicles - UAVs):



Figure 14: Athera, autonomous drone Drone (UAV) developed by Vecros, New Delhi, India.

Drones are flying robots that can be remotely operated or operate autonomously. They are used in various fields, including aerial photography, surveillance, agriculture, and package delivery.

2.2.4. Mobile Robot:

A mobile robot is capable of moving in its environment as compared to a robotic arm which is fixed or stationary at its base and other end of the manipulator with gripper maneuverers in a given workspace. Mobile robots are used in a variety of applications, from industrial automation to domestic tasks, and they rely on several technologies to navigate, interact with, and adapt to their surroundings. Some examples of mobile robots are Wheeled Robots, Legged Robots, Tracked Robots, Aerial Robots (Drones), Aquatic Robots.



Figure 15: Robomuse 5.0 developed by IIT Delhi researchers

3. Concepts in Robotics

A brief understanding of motion in one-dimension and two-dimension:

One-dimensional motion: This type of motion occurs along a straight line in one direction. Robots with one-dimensional motion can move forward or backward but cannot change direction. An example of one-dimensional motion is a linear actuator that extends and retracts in a straight line.

Two-dimensional motion: This type of motion occurs along a plane in two perpendicular directions. Robots with two-dimensional motion can move forward, backward, left, and right. They can change direction and move in multiple axes. An example of two-dimensional motion is a wheeled robot that can navigate a flat surface in any direction.

3.1. Types of motions, types of joints and links

3.1.1. Types of motions

Linear motion:

This type of motion involves movement along a straight line. Robots with linear motion can move forward and backward in a single direction. An example of a robot with linear motion is a conveyor belt system that moves objects from one point to another in a straight line.

Angular motion:

This involves rotation around an axis. Robots with angular motion can rotate parts or their entire body around a fixed point. An example of a robot with angular motion is a robotic arm that can rotate its joints to reach different positions.

Circular motion:

This type of motion involves movement along a circular path. Robots with circular motion can rotate around a central point while moving in a circular trajectory. An example of a robot with circular motion is a drone with rotating propellers that allow it to hover and move in different directions.

3.1.2. Types of joints:

Prismatic joint:

A prismatic joint is a connection between two objects that allows relative motion along a single axis. Motion that's perpendicular to this axis (or rotation about any axis) is prevented. This means the joint has one degree of freedom (1-DOF).

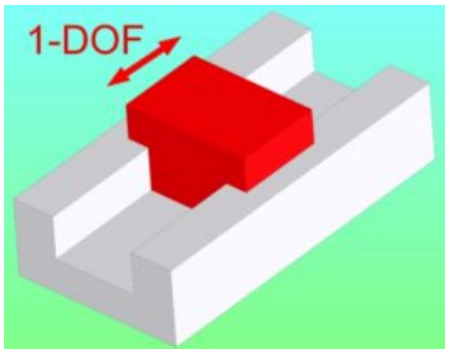


Figure 16: A Prismatic joint allows only linear motion along a single axis (9).

Revolute joint:

A revolute joint, also known as hinger joint, is a type of mechanical joint that allows rotational movement in single plane. It consists of two components, one of which rotates relative to the other along a common axis.

Revolute joint allows rotational motion about a single axis. This means the joint has one degree of freedom (1-DOF). Revolute joints are commonly used in door hinges, robotic

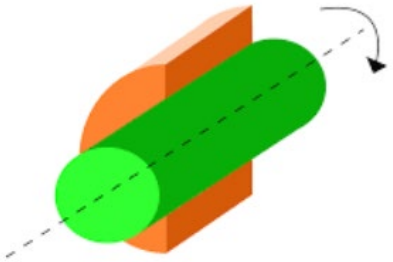


Figure 17: A revolute joint.

arms, and other applications that require rotational motion.

Spherical joint:

Spherical joints are used in applications that require a high degree of flexibility and freedom of movement. They are commonly found in humanoid robots, where they enable lifelike movements of the limbs and body.

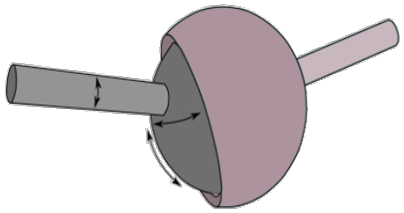


Figure 18: A spherical joint

For example in a humanoid robot designed for search and rescue missions, a spherical joint could be used at the hip. This joint would allow the robot's leg to move freely in multiple directions, providing the mobility and agility needed to navigate through complex terrains.

3.1.3. Types of links:

The combination of links and joints in a robot determines the type of motion it can perform. For example, a robotic arm with rigid links and revolute joints can replicate human arm movements, while a robot with soft links and prismatic joints might be used for tasks that require squeezing or flexing.

Rigid link:

A solid, inflexible connection between two joints. For example, the arm of a robotic crane.

Soft link:

A flexible connection that can bend and twist. For example, the trunk of a soft robot.

Relevant examples for each of the above: These concepts can be illustrated with various real-world examples, such as robotic arms in factories, humanoid robots, or soft robots designed for delicate tasks.

3.2. Using links and joints to create specific motion

When analyzing a robotic system, it's important to identify the types of links and joints used, as this will give you insight into the robot's capabilities and limitations.

Examples for demonstration: Industrial Robot Arms are a good example for demonstration. They often have a combination of prismatic and revolute joints, allowing them to move in multiple dimensions and perform complex tasks.

3.3. Degree of freedom of a robot

Definition: The degree of freedom (DOF) of a robot is the number of independent movements or parameters the robot can control. It is a measure of the robot's flexibility and capability to perform various tasks.

Identification through illustration: The DOF of a robot can be identified by analyzing its joints and links. For example, a robotic arm with three joints that can each move in one dimension would have a DOF of 3. This concept can be illustrated with various robots, such as robotic arms, drones, or humanoid robots, to show how the DOF affects the robot's ability to perform different tasks (refer to Figure 19).

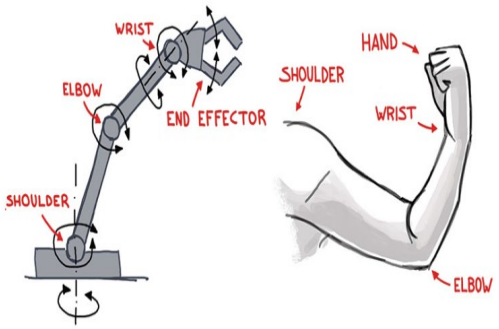


Figure 19: Degree of freedom

Artificial Intelligence

Overview of Syllabus

1. Introduction to Artificial Intelligence (AI)

(i) Meaning and brief history.

Definition of Artificial Intelligence; brief account of the history of AI since the time John McCarthy first coined the term in 1956; Turing Test, its use and importance.

(ii) Applications and Benefits of AI.

Applications of AI in different fields: commercial, industry, medical/health care, defense, banking, entertainment, transport, security and agriculture. Commonly used AI applications in daily life such as, online shopping, search engines, chatbots, voice assistants, entertainment portals, facial recognition, driver assisting vehicles, augmented/ virtual reality.

Benefits of using AI - Automation, smart decision making, assisting humans, remote patient monitoring & monitoring the progression of contagious diseases, analysis of data for research and development, efficiently solving complex problems, speedy disaster recovery strategy, performing recurring business tasks, reducing the chances of manual errors, ensuring 24-hour service availability with the same performance and consistency throughout the day.

(iii) Ethical considerations in AI.

A brief understanding of ethics in artificial intelligence including bias, prejudice, fairness, accountability, transparency, interpretability and explainability.

2. Role of Data and Information, Evolution of Computing

(i) Data and Information: Types of Data (audio, visual, numeric, text); Data to Information.

Understanding that data is pivotal to Artificial Intelligence. A brief introduction to how relevant data is identified, acquired, and explored, as a precursor to the AI Project Cycle.

(ii) Evolution of Computing: Pre AI/ML Binary Logic System, Conditional Gates, Deterministic computing for deterministic problems.

An introduction to above mentioned topics, with the emphasis that earlier computing was suited for only deterministic problems; explaining deterministic computing and deterministic problems giving relevant examples. Illustrating the limitations of deterministic computing in solving real life problems, Comparison between deterministic and probabilistic nature of real-life problems.

Note: Explanation of how AI can solve a new class of problems, based on a probabilistic paradigm. Hence Need for AI: Probabilistic, real-life problems; The AI Discretion (AI is not needed for solving deterministic problems) for example –the difference in description of temperatures by a machine and a human. A machine would make a discrete distinction between cool and hot at a given temperature for instance if 35°C is hot, then any temperature 34.9° C and below would be cool. Humans would, however, describe the temperature on a range of 'cool, pleasant, warm, hot' and so on based on their subjective experience of the temperature.



3. Introduction to Data and Programming with Python

(i) Familiarization with Python.
Introduction to Python and its elementary concepts: object-oriented, high-level, general purpose programming language. Uses and advantages of Python.

(ii) Introduction to data types and variables.
Introduction to a simple python program structure and the concept of indentation in Python, different data types in Python - numeric (int, float), Boolean, sequence type (tuple, list, strings), sets and dictionary, an understanding of what kind of data types should be used in different use cases. Introduction to variables and assignment of values.

(iii) Introduction to Operators.
Usage of different operators (arithmetic, logical, assignment, comparison, identity, membership) on data types, kind of statements which can be executed in Python.

(iv) Conditional Statements
Introduction to blocks in Python, if conditions, if else conditions, nested if conditions, if-else-if (elif) conditional block, case and switch. Shorthand conditional statements.

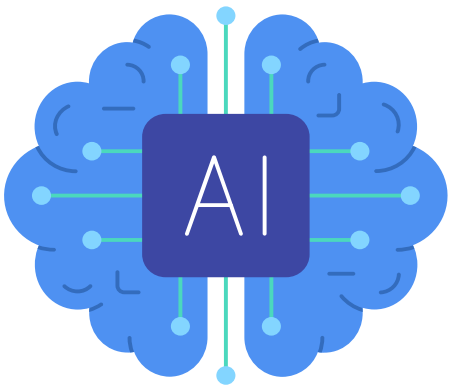
(v) Control Statements.
Meaning and use of loops in python. Different types of loops (while, for), nested loops, syntax used. 'for' loop for different types of iterables (list, tuple, string, dictionary) along with the idea of break, continue and pass statements, 'while' loop and their use cases.

(vi) Functions
An understanding of both built in and user defined functions; the importance of functions to maintain modularity; arguments given to a function (fixed and variable length); the concept of default arguments and return type of a function.

4. AI Concepts and AI Project Framework

(i) AI Concepts
Broad and narrow AI, strong versus weak AI. Expert systems in AI (for e.g., Eliza). Computer vision (CV), Natural Language Processing (NLP) and Neural Network (NN).

(ii) Components and Stages (alias AI Project Cycle).
Understanding of AI Project Framework, Stages involved in AI project: Problem Scoping, Data Acquisition, Data Exploration, Modelling and Evaluation (brief understanding of each).



1. Introduction to Artificial Intelligence

Artificial Intelligence (AI) is a remarkable field that revolves around the development of intelligent machines and computer systems. In essence, AI aims to create systems that can simulate human intelligence and perform tasks that typically necessitate human cognitive functions like learning, reasoning, problem-solving, perception, and language understanding. This chapter marks the beginning of our exploration into the world of AI, delving into its definition and the significant role it plays in our lives. The chapter includes meaning and brief history of AI, Applications of AI, Ethical considerations in AI - Bias & Fairness in AI Algorithms.



Figure 20: Significance of AI.

1.1 Meaning and brief history of AI

AI, or Artificial Intelligence, refers to the development of computer systems that can perform tasks that typically require human intelligence. These tasks include learning, reasoning, problem-solving, perception, speech recognition, and language understanding.

1.1.1. Significance of AI:

AI holds immense significance by:

- Enhancing Efficiency: Automating tasks to improve productivity.
- Enabling Innovation: Facilitating the development of advanced technologies.
- Improving Decision-Making: Analyzing vast datasets for informed choices.
- Personalizing Experiences: Customizing interactions based on user behavior.
- Addressing Complex Challenges: Tackling intricate problems with machine learning.
- Transforming Industries: Revolutionizing sectors like healthcare, finance, and manufacturing.



Figure 21: Human Robot Interaction.

1.1.2. Brief History and Evolution of AI

The journey of Artificial Intelligence (AI) stretches back through history, shaped by visionary thinkers, scientific breakthroughs, and periods of rapid advancement followed by periods of scepticism.

Table 1: History and Evolution of AI.

Year	Milestone
1950	The Theseus, a remote-controlled robotic mouse built by Claude Shannon, was able to find its way out of a labyrinth and remember its course. It is an early example of AI.
1960-1990	Steady progress in AI capabilities, including systems for playing chess, understanding natural language, and learning from data.
2010	Rapid advancements in AI, particularly in areas like image recognition, language understanding, and speech recognition, surpassing

	human performance in many standardized tests.
2014-2023	Rapid progress in AI-generated images, from primitive pixelate faces to photo realistic images based on text prompts.
2040 (projected)	Possibility of "transformative AI" being developed, with capabilities potentially matching the human brain, according to forecasts by AI researcher Ajeya Cotra.

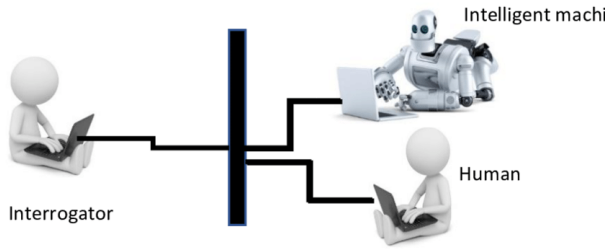


Figure 22: Turing Test.

1.2. Applications of AI

Artificial Intelligence (AI) is all around us, making our lives easier and more exciting.

1.2.1. Voice Assistants: Your AI Friends

The most popular voice assistants are Siri, Google Assistant and Alexa. These are like virtual AI friends that you can talk to! These virtual AI companions understand commands and assist with the below-mentioned features/tasks.

Setting Reminders: The voice assistant can be set to provide reminders to complete the task.

Asking Questions: The queries can be asked to find the answer.

Playing Music: The voice assistant is helpful in terms of entertainment to play music as per individual choices.

Meteorological Updates: The feature can provide the latest weather forecast.

1.2.2. Recommendation Systems: AI Understands Your Preferences

After viewing a video on YouTube, one might notice subsequent suggestions for similar content. This exemplifies AI in action. Recommendation systems function as personal advisors for entertainment and shopping preferences:

Movie Recommendations: On streaming platforms like Netflix, AI suggests movies and shows based on what you have watched before.

Music Playlists: AI can generate playlists featuring songs tailored to individual music preferences.

Online Shopping: Notice recommendations like "Customers who bought this item also purchased..."? AI assists in discovering products of potential interest.

1.2.3. Gaming and AI Opponents

AI serves not only as a helper but also as an opponent in video games. In some games, AI characters play against players, demonstrating their intelligence by adapting to actions Example:

Chess and Strategy Games: AI opponents offer challenges in games such as chess, adjusting strategies based on player moves.

Virtual Pets: AI in virtual pet games simulates real animal behavior and interacts dynamically with players.

1.2.4. Chatbots: Talking to Machines

AI helps in having a conversation with a computer.

Chatbots, AI programs designed for interaction, fulfill diverse roles Example:

Customer Support: The chatbot assists a website in providing answers to the raised queries.

Language Learning: It helps in learning a new language by chatting and practicing.

1.2.5. AI in Healthcare

AI is transforming healthcare in amazing ways:

Diagnosis Assistance: AI helps doctors analyse medical images like X-rays to identify diseases.

Drug Discovery: AI can aid in predicting drugs' bioactivity by leveraging machine learning algorithms to analyse vast amounts of data on known compounds and their biological activities.

Personalized Treatment: AI suggests personalized treatment plans based on patients' medical history and genetics.



Figure 23: AI in Finance.

1.2.6. AI in Finance sector

- Algorithmic Trading: AI algorithms predict market trends and help traders make decisions.
- Fraud Detection: AI identifies unusual patterns in transactions to catch fraudulent activities.
- Customer Service: AI chatbots assist customers with account inquiries and financial advice.

1.3. Ethical Considerations in AI - Bias and Fairness in AI Algorithms

Artificial Intelligence has the potential to do a lot of good, but it can also raise ethical concerns. One of the most significant concerns is bias and fairness in AI algorithms.

1.3.1. Bias in AI Algorithms

What is Bias in AI?

Bias in AI refers to the presence of unfair and prejudiced behaviour in the decisions made by AI systems. AI algorithms, like humans, can develop biases based on the data they are trained on. These biases can lead to unfair treatment of certain groups or individuals.

How does bias happen in AI?

Imagine you're teaching an AI to recognize whether people are doctors or not based on their photos. If you mostly show it pictures of male doctors, it might mistakenly think that doctors are mostly male. This is an example of bias. Bias can also happen when the data used for training contains unfair stereotypes or is not representative of the real world.

Consequences of Bias in AI:

Unfair Treatment: AI systems can make unfair decisions, such as denying loans or job opportunities to certain groups.

Reinforcing Stereotypes: Bias can perpetuate harmful stereotypes, which is not only unfair but also harmful to society.

Legal and Ethical Issues: Biased AI systems can raise legal and ethical concerns, potentially leading to lawsuits, regulatory scrutiny, and reputational damage for organizations deploying these systems.

Inaccurate Decisions: Bias can result in inaccurate predictions or decisions, such as denying opportunities or services to deserving individuals or groups.

1.3.2. Fairness in AI Algorithms

What is Fairness in AI?

Fairness means that AI systems treat all individuals or groups fairly and without discrimination. Fair AI algorithms aim to provide equal opportunities and outcomes to everyone.

How Can AI Be Fair?

AI fairness involves making sure that the AI:
-Does not discriminate against any group.
-Is transparent in the decision-making process.
-Is testing to validate fairness in its choices.



Figure 24: AI Ethics.

1.3.3. Interactive Activity: AI Ethics Discussion

Now, let's dive into an interactive discussion about AI ethics. Some ethical dilemmas related to AI will be presented for the group to discuss and decide on the best course of action.

Ethical Dilemma 1: AI Hiring

Imagine a company using AI to screen job applications. However, the AI has learned from historical hiring data, which might have contained biases. As a result, it tends to reject applications from a certain gender. What should the company do to address this issue?

Ethical Dilemma 2: Predictive Policing

A city uses AI to predict where crimes are likely to happen. However, this AI system ends up patrolling certain neighbourhood more often than others, leading to more arrests in those areas. Is this fair? How can the city ensure fairness in its policing AI?

2. Role of Data and Information, Evolution of Computing

In this chapter, we will explore the vital role of data and information in Artificial Intelligence (AI) and understand how data is crucial for AI projects. We'll also briefly touch on the evolution of computing.

2.1 Data and Information

2.1.1. Types of Data:

Data comes in various forms, each valuable for different AI tasks:

Audio Data: This includes sounds, music, and spoken words, which can be used for speech recognition and audio analysis.

Visual Data: Images and videos are visual data. AI can analyse them for tasks like object detection, facial recognition, and image classification.

Numeric Data: Numeric data consists of numbers and measurements, often used in statistical analysis and predictive modelling.

Text Data: Textual data includes written words and documents. Natural Language Processing (NLP) techniques are applied to analyse and understand text data.

2.1.2. Data to Information:

Data becomes valuable when it is processed, analysed, and transformed into meaningful information. AI algorithms are designed to extract insights, patterns, and knowledge from data, converting it into actionable information.

Aspect	Data	Information
Nature and Form	Raw, unorganized facts, figures, symbols	Processed, organized, structured, meaningful context
Purpose	Input for generating information	Output of processed data, aids decision-making
Representation	Numbers, text,	Reports, summaries,
Usage and Interpretation	Requires processing for insights images, symbols	Already processed and interpretable, charts, graphs
Context and Meaning	Lacks context and meaning until processed	Provides context, relevance, purpose

DIKW MODEL

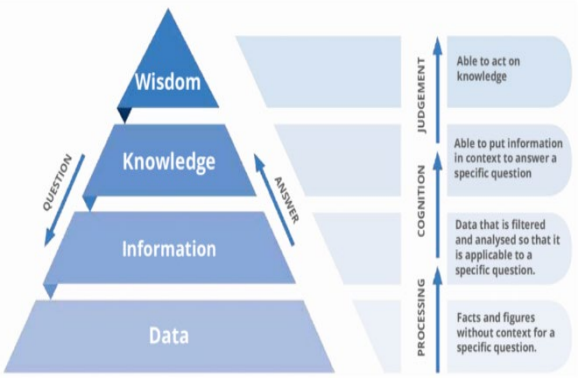


Figure 25: DIKW model.

The DIKW model, which stands for Data, Information, Knowledge, and Wisdom, is a framework often used to describe the hierarchy of information processing and understanding. While it's not inherently an AI-specific model, it's relevant to AI because AI systems often deal with processing data into higher forms of understanding. Here's a breakdown of each level:

Data: Data refers to raw, unorganized facts or observations. Its the most basic level of information and lacks context or meaning on its own. For example, a list of numbers, words, or symbols without any interpretation or analysis represents data.

Information: Information is data that has been processed, organized, or structured in some way to make it meaningful. It provides context or relevance to the data. This could involve

categorizing, sorting, summarizing, or filtering the data to extract useful insights. For instance, arranging a list of numbers in ascending order or grouping similar items together turns raw data into information.

Knowledge: Knowledge involves understanding and interpreting information within a specific context. It goes beyond the mere accumulation of facts and involves comprehension, analysis, and synthesis. Knowledge is acquired through experience, learning, and reasoning. In the context of AI, knowledge might involve recognizing patterns, making inferences, or drawing conclusions from the information provided.

Wisdom: Wisdom is the highest level of understanding and involves the ability to make sound judgments, decisions, and choices based on knowledge and experience. It requires a deep understanding of underlying principles, values, and long-term consequences. Wisdom often involves the application of ethical considerations and a broader perspective beyond immediate circumstances.

In AI, the goal is often to progress from data to wisdom through various computational techniques such as data mining, machine learning, natural language processing, and knowledge representation. AI systems aim to not only process and analyze data but also to extract meaningful information, derive knowledge, and ultimately make intelligent decisions or recommendations akin to human wisdom.

2.1.3. Identifying Relevant Data for AI

Acquiring Data:

Data can be collected from various sources such as sensors, websites, databases, or user interactions.

Exploring Data:

Data exploration involves understanding the data's structure, relationships, and potential insights. Techniques like data visualization and statistical analysis help in data exploration.

2.2. Evolution of Computing

2.2.1. From Mainframes to AI:

Computing has evolved from room-sized mainframes to powerful and compact devices. AI's development was greatly facilitated by advances in computing power and data storage.

AI Project Cycle Precursor:

Understanding the evolution of computing and data's role is essential as a precursor to the AI project cycle. Computing advancements enable AI algorithms to process large datasets and make complex decisions.

2.2.2. Evolution of Computing: Pre AI/ML

Binary Logic System:

The foundation of modern computing lies in binary logic, using 0s and 1s to represent data. Binary logic allows computers to process and manipulate data efficiently through electronic switches.

Conditional Gates:

In early computing, logical operations were performed using conditional gates such as AND, OR, and NOT.

These gates allowed for decision-making and data processing based on logical conditions.

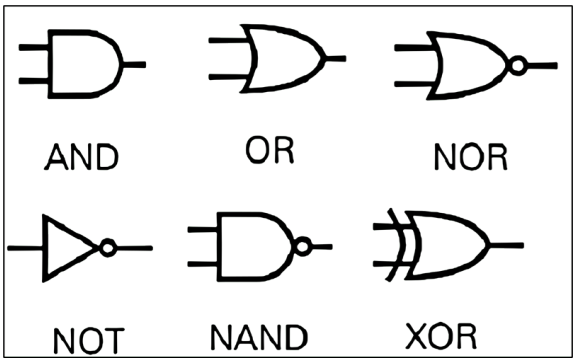


Figure 26: Logic gates.

Deterministic Computing for Deterministic Problems:

Early computers were primarily designed for deterministic problems, where the outcome is entirely determined by the input and operations. These computers were used for

tasks like numerical calculations, data processing, and solving equations.

2.2.3. Relevance to AI and ML

Foundation for AI and ML:

The evolution of computing, especially the development of binary logic and conditional gates, laid the foundation for AI and ML.

Transition to Probabilistic Problems:

While early computing was suited for deterministic problems, AI and ML often deal with probabilistic problems where outcomes are uncertain.AI and ML algorithms use statistical methods to make predictions and decisions based on data.

2.2.4. Understanding the Transition

Computing Paradigm Shift:

Understanding the transition from deterministic computing to probabilistic AI/ML is crucial for appreciating the complexity of modern AI systems.

AI and ML algorithms can handle uncertainties and make informed decisions in dynamic, real-world scenarios.



3. Introduction to Data and Programming with Python

This section provides a comprehensive introduction to Python, a versatile, object-oriented, high-level programming language widely known for its simplicity and readability. The exploration includes fundamental characteristics, practical applications, and the advantages Python offers.

3.1. Familiarization with Python:

3.1.1. Introduction to Python

- Python is a versatile and widely-used programming language, prized for its simplicity and readability.
- It's an object-oriented language, allowing the creation and manipulation of objects, mirroring real-world entities.
- Python's high-level nature abstracts low-level details, making it beginner-friendly and closer to human language.
- Designed as a general-purpose language, Python spans web development, scientific computing, data analysis, and artificial intelligence.

3.1.2. Uses and Advantages of Python:

- Versatility: Widely employed in web development, data science, and AI using frameworks like Django, Flask, NumPy, Pandas, Scikit-Learn, and TensorFlow.
- Readability: Python's clean syntax reduces code complexity, enhancing comprehension, and maintainability, particularly beneficial for beginners.
- Large Community: Python enjoys a vast and active developer community, providing robust support, resources, and numerous libraries.
- Cross-Platform: Python is platform-independent, running seamlessly on Windows, macOS, and Linux.

Example: Write a python program to print " Hello World".

Input

```
1  print("Hello World")
```

Output:

HELLO WORLD

3.2. Introduction to Data Types and Variables

3.2.1. Introduction:

In this section, we will dive deeper into Python programming by exploring data types, variables, and the structure of Python programs. Students will learn about various data types in Python, how to declare variables, and understand the importance of selecting the appropriate data types for different use cases.

3.2.2. Data Types in Python:

- Numeric Types: Python supports integer (int) and floating-point (float) numbers for mathematical operations.
- Boolean Type: Boolean (bool) represent truth values, either True or False.
- Sequence Types: These include tuples (tuple), lists (list), and strings (str). Tuples and lists store ordered collections of items, while strings store text.
- Sets: Sets (set) are unordered collections of unique items.
- Dictionaries: Dictionaries (dict) store key-value pairs, allowing efficient data retrieval.



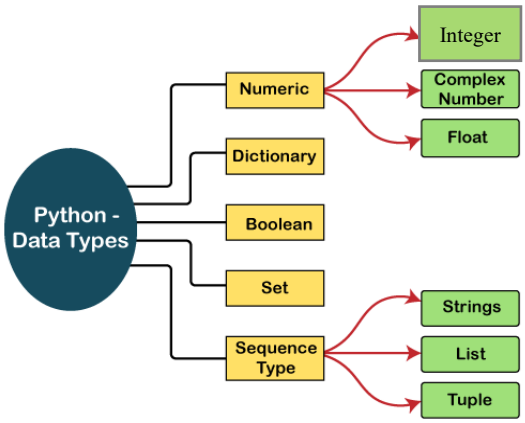


Figure 27: Python Data types.

3.2.3. Selecting the Right Data Type:

The choice of data type depends on the nature of the data and the operations you want to perform. For example, use integers for counting, floating-point for measurements, and lists for collections of items.

3.2.4. Introduction to Variables:

Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

Example:

```
x = 5
y = "John"
print(x)
print(y)
```

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the Python keywords.

Python Program Structure:

- Python programs are structured with proper indentation.

  Example: Write a Python program that demonstrates the usage of different data variables

```
# Integer variable
num = 10
print("Integer variable:", num)

# Float variable
float_num = 3.14
print("Float variable:", float_num)

# String variable
name = "John"
print("String variable:", name)

# Boolean variable
is_true = True
print("Boolean variable:", is_true)

# List variable
my_list = [1, 2, 3, 4, 5]
print("List variable:", my_list)

# Tuple variable
my_tuple = (1, 2, 3, 4, 5)
print("Tuple variable:", my_tuple)

# Dictionary variable
my_dict = {'name': 'John', 'age': 30, 'city': 'New York'}
print("Dictionary variable:", my_dict)

# Set variable
my_set = {1, 2, 3, 4, 5}
print("Set variable:", my_set)
```

Output

```
Integer variable: 10
Float variable: 3.14
String variable: John
Boolean variable: True
List variable: [1, 2, 3, 4, 5]
Tuple variable: (1, 2, 3, 4, 5)
Dictionary variable: {'name': 'John',
'age': 30, 'city': 'New York'}
Set variable: {1, 2, 3, 4, 5}
```

3.3. Introduction to Operators

3.3.1. Introduction:

In this section, we will introduce students to the concept of operators in Python, which are essential for performing various operations on data.

3.3.2. Types of Operators:

- Arithmetic Operators: These include addition (+), subtraction (-), multiplication (*), division (/), modulus (%), and exponentiation (**). They are used for basic mathematical operations.
- Logical Operators: Logical operators like and, or, and not are used for combining conditions and making logical decisions in Python.
- Assignment Operators: Assignment operators (=, +=, -=) are used to assign values to variables and modify their values.
- Comparison Operators: These operators (==, !=, <, <=, >, >=) are used to compare values and return Boolean results.
- Identity Operators: Identity operators (is and is not) are used to compare object identity.
- Membership Operators: Membership operators (in and not in) check for the presence of a value in a sequence (e.g., a list).

3.3.3. Usage of Operators:

Operators are applied to different data types to perform various operations, such as arithmetic operations on numbers, logical operations on Boolean values, and membership checks in sequences.

Example: Write a python program that demonstrates the usage of different operators:

```
# Arithmetic operators
a = 15.1
b = 5
print("Arithmetic operators:")
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
# Comparison operators
x = 10
y = 20
print("\nComparison operators:")
print("Equal to:", x == y)
print("Not equal to:", x != y)
print("Greater than:", x > y)
print("Less than:", x < y)
print("Greater than or equal to:", x >= y)
print("Less than or equal to:", x <= y)
# Logical operators
p = True
q = False
print("\nLogical operators:")
print("AND:", p and q)
print("OR:", p or q)
print("NOT:", not p)

# Assignment operators
c = 5
print("\nAssignment operators:")
c += 3  # equivalent to c = c + 3
print("Addition assignment:", c)
c -= 2  # equivalent to c = c - 2
print("Subtraction assignment:", c)
c *= 4  # equivalent to c = c * 4
print("Multiplication assignment:", c)
c /= 2  # equivalent to c = c / 2
print("Division assignment:", c)

# Identity operators
list1 = [1, 2, 3]
list2 = [1, 2, 3]
print("\nIdentity operators:")
print("Is:", list1 is list2)
print("Is not:", list1 is not list2)

# Membership operators
print("\nMembership operators:")
print("In:", 1 in list1)
print("Not in:", 4 not in list1)

```



Output

```
Arithmetic operators:
Addition: 20.1
Subtraction: 10.1
Multiplication: 75.5
Division: 3.02
Floor Division: 3.0
Modulus: 0.09999999999999964
Exponentiation: 785027.2575099999

Comparison operators:
Equal to: False
Not equal to: True
Greater than: False
Less than: True
Greater than or equal to: False
Less than or equal to: True

Logical operators:
AND: False
OR: True
NOT: False

Identity operators:
Is: False
Is not: True

Membership operators:
In: True
Not in: True
```

3.3.4. Types of Statements:

Python supports various types of statements, including:
Assignment Statements: Used to assign values to variables.
Conditional Statements: Such as if, else, and elif, for decision-making.
Looping Statements: Such as for and while, for repetitive tasks.
Print Statements: Used to display output.
Comment Statements: Comments are preceded by # and are used for documentation and explanations.

3.4. Conditional Statements

3.4.1. Introduction:

In this section, we'll explore conditional statements in Python, which allow us to make decisions in our programs. We'll cover various aspects of conditional statements, including if conditions, if-else conditions, nested if conditions, if-else-if (elif) conditional blocks, and shorthand conditional statements.

3.4.2. Conditional Statements:

Conditional statements allow us to execute different code blocks based on specified conditions.

3.4.3. Introduction to Blocks in Python:

Python uses indentation to define blocks of code, making it easy to read and maintain.

If Conditions:

- The if statement is used to execute a block of code if a condition is true.
- Example: if x > 10:

If-Else Conditions:

- The if-else statement is used to execute one block of code if a condition is true and another block if the condition is false.
- Example: if x > 10: ... else: ...

Nested If Conditions:

- You can have an if statement inside another if statement. This is called nested if.
- Example: if x > 10: ... if y < 5: ...

If-Else-If (Elif) Conditional Block:

- The elif statement allows you to check multiple conditions in sequence.
- It is used when you have multiple options and only one of them should execute.
- Example: if x > 10: ... elif x < 5: ... else: ...

Case and Switch (Note):

- Unlike some other languages, Python does not have a case or switch statement. Instead, it relies on if-elif-else for similar functionality.



Shorthand Conditional Statements:

- Python allows for concise conditional expressions using the ternary operator.
- Example: result = "Yes" if condition else "No"

  Example: Write a python program using "if – else" statement

```
# This program checks if a number is positive, negative, or zero

# Input: User provides a number
number = int(input("Enter a number: "))

# If-else statement to check the number
if number > 0:
    print("The number is positive.")
elif number < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

Output:

```
Enter a number: 10
The number is positive.
```

3.5. Control Statements and Loops

3.5.1. Introduction:

In this section, we'll delve into control statements and loops in Python. These statements allow us to control the flow of our programs, repeat actions, and iterate through data structures. We'll cover the meaning and use of loops, including the while and for loops, nested loops, and introduce concepts like break, continue, and pass statements.

(a) ***Control Statements:***

- Control statements are used to determine the flow of a program.
- They include loops and conditional statements (which were covered in the previous section).

(b) ***Loops in Python:***

- Loops are used to repeatedly execute a block of code as long as a certain condition is met.
- Python provides two main types of loops: while and for.

(c) ***The for Loop:***

- The for loop is used to iterate over a sequence (such as a list, tuple, string, or dictionary) or other iterable objects. It executes a block of code for each item in the sequence.

(d) ***The while Loop:***

- The while loop is used when a code block must be executed repeatedly if a condition is true.
- *Syntax*: while condition:

(e) ***Nested Loops:***

- You can have one or more loops inside another loop, forming nested loops.
- Useful for complex iterations or working with multi-dimensional data structures.

(f) ***Break, continue, and pass Statements:***

- break: Used to exit a loop prematurely when a certain condition is met.
- continue: Skips the current iteration and continues with the next iteration of the loop.
- pass: Placeholder statement that does nothing. Used when a statement is syntactically required but no action is needed.

3.6. Functions

3.6.1. Introduction:

In this section, we'll explore the concept of functions in Python. Functions are reusable blocks of code that perform specific tasks, enhancing code modularity and readability.

3.6.2. Functions in Python:

Functions are reusable blocks of code that perform specific tasks when called.
They help break down complex programs into smaller, manageable parts.



(a) ***Built-in Functions:***

- Python provides a wide range of built-in functions (e.g., **print()**, **len()**, **input()**) that perform common tasks.
- These functions are available without the need for additional code.

(b) ***User-Defined Functions:***

- Users can define their functions to perform custom tasks.
- User-defined functions improve code modularity, making it easier to maintain and debug.

(c) ***Function Arguments:***

- Functions can receive data via arguments. There are two types:
- Fixed-Length Arguments: The number of arguments is fixed when defining the function.
- Variable-Length Arguments: Functions can accept a variable number of arguments.

(d) ***Default Arguments:***

- Functions can have default values assigned to some or all of their arguments.
- Default arguments are used when no value is provided for a specific argument.

(e) ***Return Type:***

- Functions can return a value (or multiple values) to the caller.
- The return type specifies the type of data the function will return.

3.6.3. Introduction to Python Dictionaries:

Dictionaries are used to store data values in key: value pairs.

A dictionary is a collection which is ordered, changeable and do not allow duplicates.

Create and print a dictionary:

Example

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Dictionary Length

To determine how many items a dictionary has, use the len() function:

Print the number of items in the dictionary

```
print(len(thisdict))
```

Dictionary Items

Dictionary items are ordered, changeable, and do not allow duplicates.

Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

Example

Print the "brand" value of the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])
```

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

Example

String, int, boolean, and list data types:

```
thisdict = {
  "brand": "Ford",
  "electric": False,
  "year": 1964,
  "colors": ["red", "white", "blue"]
}
```



Example: Write a python program to create and print a dictionary

```
# This program creates and prints a dictionary

# Creating a dictionary with some sample data
my_dictionary = {
    "name": "Alice",
    "age": 25,
    "city": "New York",
    "occupation": "Engineer"
}

# Printing the dictionary
print("The dictionary is:")
print(my_dictionary)

# Optionally, you can print each key-value pair separately
print("\nKey-Value pairs in the dictionary:")
for key, value in my_dictionary.items():
    print(f"{key}: {value}")
```

Output:

```
The dictionary is:
{'name': 'Alice', 'age': 25, 'city':
New York', 'occupation': 'Engineer'}

Key-Value pairs in the dictionary:
name: Alice
age: 25
city: New York
occupation: Engineer
```



4. AI Concepts and AI Project Framework

Artificial Intelligence (AI) encompasses a wide spectrum of capabilities, ranging from specialized task-solving to human-level cognitive understanding. Understanding these categories is crucial for comprehending the current state and potential future of AI systems.

4.1 AI concepts

4.1.1. Narrow AI (Weak AI)

Narrow AI, or Weak AI, is designed for specific tasks without the ability to generalize knowledge across domains. It excels within defined scopes but lacks broader adaptability.

Examples:

Speech Recognition:
Usage: Virtual assistants (Siri, Google Assistant, Alexa).
Function: Understand spoken language, provide relevant information.

Key Characteristics:

- Task-specific intelligence.
- Limited adaptability outside predefined contexts.
- Common in daily applications like virtual assistants and recommendation systems.

4.1.2. General AI (Strong AI)

Theoretical AI with human-like cognitive abilities, contrasting with task-specific narrow AI.

Key Characteristics:

- Autonomous Learning: Rapid adaptation without explicit programming.
- Common Sense Reasoning: Intuitive decision-making across scenarios.
- Creativity and Innovation: Generating new ideas and solutions.
- Human-Like Understanding: Comprehending emotions, social dynamics, and context.

Key Points:

- Hypothetical AI mirroring human intelligence.
- Exhibits versatility, autonomy, and cognitive breadth.

4.1.3. The Journey Ahead

While narrow AI has made significant strides and is widely employed, the development of general AI presents profound technical and ethical challenges. Creating a machine with human-level cognitive abilities involves replicating complex human thought processes, self-awareness, and adaptability. Moreover, ethical considerations like AI safety, bias, and control must be addressed to ensure responsible development.



Figure 28: Virtual Assistant.

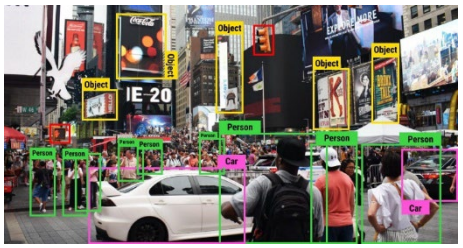


Figure 29: Image Recognition.



Figure 30: Automated Driving.

4.1.4. Examples of AI Applications

(a) Virtual Assistants:

- Siri (Apple), Google Assistant (Google), Cortana (Microsoft).

(b) Image Recognition:

- Facebook's Automatic Tagging, Google Photos, Security and Surveillance.

(c) *Healthcare Diagnostics:*

- Radiology Imaging, Diabetes Management, Drug Discovery etc.

(d) *Autonomous Vehicles:*

- Tesla Autopilot, Waymo (Google), Delivery Drones.

(e) Financial Services:

- Algorithmic Trading, Fraud Detection, Customer Service Chatbots.

4.1.5. How AI Works - Basics of Machine Learning

Basics of Machine Learning:

(a) Data:

- Foundation for machine learning.
- Examples include images, text, and numerical values.

(b) Training Process:

- AI learns patterns and relationships in data.
- Adjusts internal parameters iteratively.

(c) Model Creation:

- System creates a representation of learned patterns.
- Encapsulates traits distinguishing different categories.

(d) Prediction/Inference:

- Trained model makes predictions on new data.
- Applies learned patterns to produce accurate outputs

4.1.6. Types of Machine Learning:

(a) *Supervised Learning*: In supervised learning, the AI system is provided with labelled data, meaning the desired output is known for each input.

(b) *Unsupervised Learning*: Unsupervised learning involves working with unlabelled data. The AI system's task is to find patterns, similarities, or groupings within the data.

(c) *Reinforcement Learning*: In reinforcement learning, the AI system learns by interacting with an environment. It receives feedback in the form of rewards or penalties based on its actions.

4.1.7. Applications of Machine Learning:

Machine learning finds application in a diverse array of fields. Few are listed below:

(a) *Healthcare*: Diagnosing diseases from medical images, predicting patient outcomes, and recommending personalized treatments.

(b) *Finance*: Detecting fraudulent transactions, predicting stock prices, and credit risk assessment.

(c) *Natural Language Processing (NLP):* Sentiment analysis of social media posts, language translation, and chatbots that interact with users.

4.1.8. Difference between AI, Machine Learning, and Neural Networks

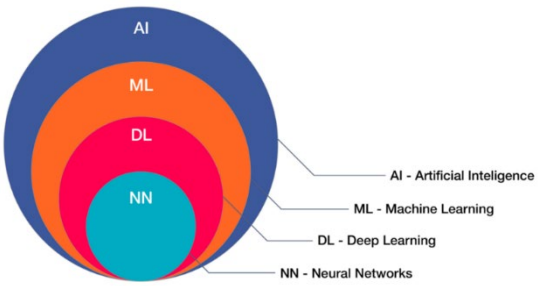


Figure 31: Difference between AI , ML , DL and NN.

Relationships:

AI is the overarching field that aims to create intelligent machines.

ML is a subset of AI that provides techniques for machines to learn from data.

Neural networks are a subset of ML algorithms that model complex patterns and relationships in data.

In essence, AI sets the vision of creating intelligent machines, ML provides the tools for machines to learn from data, and neural networks are a specific technique within ML that enable machines to process and understand complex information. These concepts work together to drive innovation and advancement in technology, transforming how we interact with the digital world.

4.1.9. Basics of Neural Networks

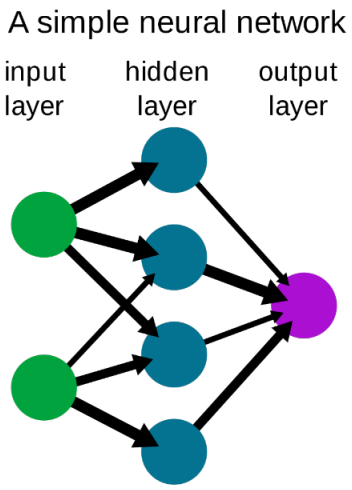


Figure 32: Simple Neural Network.

Neural networks are a fundamental component of machine learning, designed to simulate the workings of the human brain. This chapter delves into the intricacies of neural networks, exploring their structure, functions, and role in enabling machines to process complex data and make predictions.

What are Neural Networks?

Neural networks, often referred to as artificial neural networks (ANNs), are a class of algorithms inspired by the biological neural networks in the human brain. These algorithms are designed to recognize patterns, solve problems, and make decisions by learning from data. Neural networks consist of interconnected nodes, or artificial neurons, organized into layers.

Artificial Neurons and Activation Functions

Each node in a neural network layer is a mathematical abstraction of a neuron in the human brain. These nodes receive inputs, apply mathematical operations, and produce an output. The output of a node is determined by an activation function, which introduces non-linearity to the network.

4.2. Components and Stages (alias AI Project Cycle).

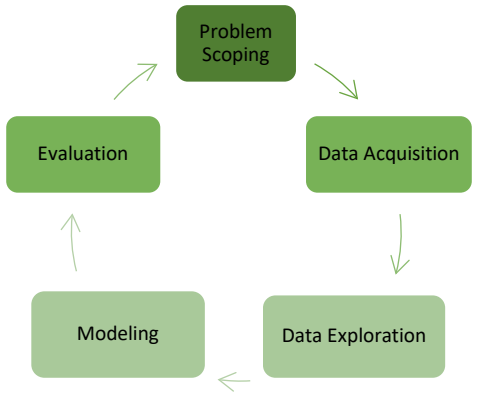


Figure 33: Different components and stages

The AI project cycle typically consists of these five stages:

1. Problem Scoping
2. Data Acquisition
3. Data Exploration
4. Modeling
5. Evaluation

Problem Scoping: Problem scoping is the process of pinpointing a particular issue or opportunity that can be tackled using artificial intelligence (AI). During this phase, we not only identify the problem but also set specific objectives, goals, and criteria for success. However, scoping a problem is no simple task. It requires a deep understanding of the issue so that we can work effectively and solve problem-solving.

***Data Acquisition*:** This is the second phase of the AI project cycle, which is focused on obtaining the necessary data for the project. While developing an AI system for predictive



purposes, it's essential to begin by training it with relevant data.

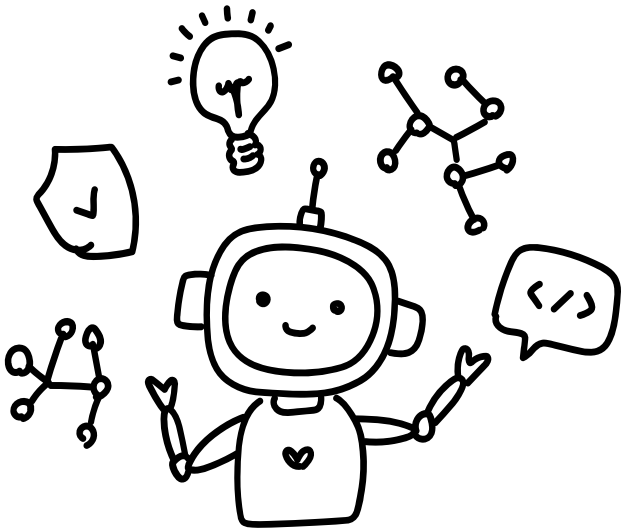
***Data Exploration*:** Data is a complicated thing, often just a bunch of numbers. But to make sense of it, we need to find the hidden patterns. That's where data visualization comes in. It's all about turning those numbers into pictures that are easy for people to understand.

***Modeling**:* In the AI project cycle, modeling is a critical step in simplifying complex data for computers to process and make predictions. At the start, data is usually presented in charts or graphs to help people spot patterns. But, for AI systems to work, we need to convert this data into a basic form that computers can grasp that is binary (0s and 1s).

Evaluation: This is the last stage of the AI project cycle. Once you've created and trained a model, it's crucial to thoroughly test it to evaluate how well it performs.



IV. SYLLABUS CLASS X



Robotics

Overview of Syllabus

1. New Age Robotic Systems

Warehouse Robots, Assistant Robots, Smart Homes, Smart Schools, Smart mobility, Autonomous Cars/Driver Assisted Cars, Autonomous Drones, Robotics for Medicine and Healthcare, any other (give examples). Why NARS are relevant and possible. Some common examples of robotic systems such as, elevator.

2. From Robots to Cobots

(i) Difference between a machine and a robot.

Difference between a machine and a robot: an understanding of how a robot must continuously sense, maintain a certain speed and do a certain task that it is programmed to do, using examples; identification of machine/robot through illustrations.

(ii) Cobots

Meaning of Cobots (as robotic systems with humans in the loop, emphasizing the provision for human interaction and a multi robot system); difference between Cobot and Robot, importance of cobots, a brief understanding of the progress from Robots to Cobots.

3. Components of Robots as a System

(i) Introduction to Gears.

Use of gears in robots- transmission and amplification of force, principles of gears, types of gears, gear ratio (brief understanding with examples).

(ii) Sensors in Robotics.

Types of Sensors based on application (Vision, tactile, temperature, range and proximity detection, motion, navigation, speech recognition) with examples only.

Classification of sensors as Internal Sensors (Position, Velocity, Acceleration and Force Sensors) and External Sensors (Contact type, Non-contact Type Sensors). Functions of sensors.

(iii)Actuators

Brief understanding of actuators and their application; types of actuators (linear and rotary) with examples such as joints and wheels.

(iv)Controller for a Robotic System

Examples of control systems from daily life: regulation of fan, refrigerator, and air conditioner.

Control systems in robotics: Meaning, functions and working; differences between manual and automatic control systems. Block diagrams to be used to illustrate (Input-controller-robot-feedback).

(v) Integrating Sensors, Actuators and Controller in a Robotic System

Illustration using a simple example: how angular position is measured by a position sensor of a robotic arm is controlled by driving the revolute joint using a motor.



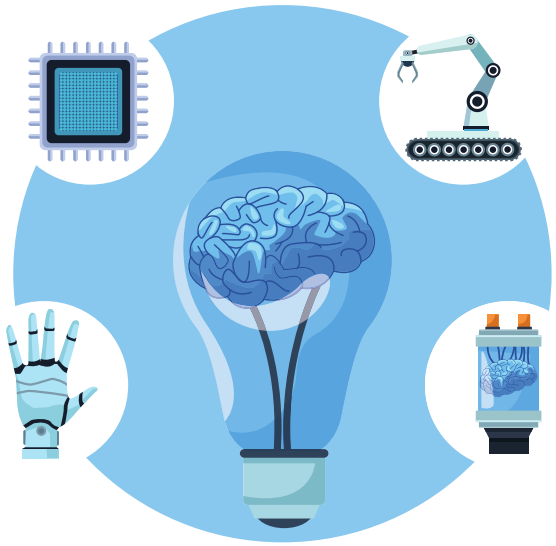
4. Visualization, Design and Creation of Components

(i) Application of Mechanical Block of Robotics.
Visualize, design and create components of a robot.
Using Tinkercad to visualize, design, and create the components of a robot; the different types of joints: revolute and prismatic, RR Mechanism.

(ii) Visualization of motion.
Use of Tinkercad to visualize motion of the components designed.

5. Integrating Robots as a System

Building simple robotic systems, wheeled mobile robot, Single Board Computer coding
Using Tinkercad to build simple robotics systems, for example, RR Mechanism. Building simple systems up to a mobile robot with four wheels.



1. New Age Robotics Systems (NARS) Overview

The New Age Robotic Systems (NARS) refers to the transformation of the traditional practices in Robotics through innovative approaches using cutting-edge technologies to create more sophisticated versatile and intelligent robotic systems. The following are some of the applications:

1.1. Warehouse Robots:
- Automate inventory handling.
- Example: Amazon's Kiva robots.

1.2. Assistant Robots:
- Aid in daily tasks.
- Example: Roomba robot vacuum.

1.3. Smart Homes:
- Integrate robotics for comfort.
- Example: Samsung's Bot Handy.

1.4. Smart Schools:
- Use robots for teaching.
- Example: NAO humanoid robot.

1.5. Smart Mobility:
- Enhance transportation with tech.
- Example: Waymo's autonomous vehicles.

1.6 Autonomous Cars:
- Operate without human intervention.
- Example: Tesla's Autopilot.

1.7 Autonomous Drones:
- Fly and perform tasks autonomously.
- Example: DJI's Mavic drones.

1.8 Healthcare Robotics:
Assist in surgery and patient care.
Example: da Vinci Surgical System.

Note: Explanation of each topic can be a classroom activity.

Relevance and Possibility of NARS:
- **Relevance:** Revolutionize industries, enhance efficiency.
- **Possibility:** Enabled by tech advancements.



2. From Robots to Cobots

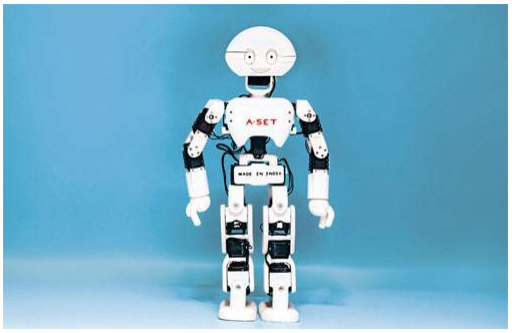
2.1 Difference between a machine and a robot

Feature	Robot	Machine
Autonomy	Operates independently, with minimal human intervention	Requires human operation or programmed to follow a fixed set of instructions
Intelligence	May have varying degrees of intelligence, from basic programming to advanced AI	Typically lacks intelligence or has very limited decision-making capabilities
Mobility	Can move around and interact with the physical world	Stationary or has limited movement within a defined space
Sensors	Often has sensors to perceive its environment	May have sensors for specific inputs, but not as comprehensive as a robot
Adaptability	Can adapt to changing situations within its programmed parameters	Limited adaptability, relies on pre-programmed responses
Complexity	More complex due to the need for autonomy, intelligence, and mobility	Less complex, focusing on specific tasks
Examples	Industrial robots, surgical robots, autonomous vehicles	Washing machine, calculator, blender

2.2 Cobot

"Cobot" stands for Collaborative Robot. They are robotic systems designed to work alongside humans in a shared workspace, emphasizing interaction and collaboration. Unlike traditional industrial robots, cobots operate without physical barriers and prioritize safety for human interaction. Cobots often function as part of a multi-robot system, collaborating with each other and humans to achieve common goals. This allows for flexible and adaptable automation, especially in dynamic environments.

Example: MANAV Humanoid Robot: Initiative developed by the Indian Institute of Technology (IIT) Bombay. It is designed to be a versatile and adaptable robot capable of performing various tasks and functions.



2.2.1 Difference between Cobot and Robot:

Feature	Cobots	Robots
Collaboration	Designed to work alongside humans	Operated separately from humans
Safety Features	Equipped with sensors and collision detection	Often enclosed in safety cages
Flexibility	More flexible and easier to reconfigure	Less flexible and more specialized

Applications	Used in collaborative tasks across various industries	Commonly used for repetitive tasks in industrial settings

2.2.2 Importance of Cobots:

- Increased productivity and efficiency: Cobots can handle repetitive tasks while humans focus on complex ones, leading to faster production cycles.
- Improved worker safety: By taking over hazardous tasks, cobots reduce the risk of injuries for human workers.
- Enhanced flexibility and adaptability: Cobots can easily adapt to changing production demands and new tasks, making them suitable for dynamic environments.

2.2.3 *Progress from Robots to Cobots:*

Early robots focused on speed, precision, and power were designed to operate autonomously in closed environments. However, concerns about safety and the need for flexible automation led to the development of collaborative robots.

Cobots emphasize safety, ease of use, and adaptability, allowing for human-robot collaboration in shared workspaces. This shift reflects a growing understanding of the benefits of human-robot collaboration.



3. Components of Robots as a System

Prerequisite: Refer to Chapter 2 of Grade 9

3.1. Introduction to Gears

Gears are used in robots for the transmission and amplification of force.

Types of gears: There are several types of gears, including spur gears, bevel gears, worm gears, and planetary gears. Each type has its own set of characteristics and is used for specific applications.

Gear ratio: The gear ratio is the ratio of the number of teeth on the output gear to the number of teeth on the input gear.

Example: If an input gear with 10 teeth is connected to an output gear with 20 teeth, the gear ratio is 1:2.

3.2. Sensors in Robotics

A sensor is a device whose purpose is to sense (to detect) some characteristic of its environments. It detects events or changes in quantities and provides a corresponding output, generally as an electrical or optical signal.

3.2.1. Common types of sensors in robotics include:

Ultrasonic Sensors: Measure distances using sound waves, useful for obstacle detection.

Infrared Sensors: Detect heat or proximity, often used for line-following in mobile robots.

LIDAR (Light Detection and Ranging): Provides precise 3D mapping and distance measurements, commonly used in autonomous vehicles.

Tactile sensors: Used to detect touch or pressure. Example: Pressure-sensitive pads.

Temperature sensors: Used to measure temperature. Example: Thermocouples.

Motion sensors: Used to detect movement. Example: Accelerometers.

Navigation sensors: Used to determine the position and orientation of the robot. Example: GPS.

Speech recognition sensors: Used to interpret spoken commands. Example: Microphones.

Cameras: Used for visual perception, object recognition, and navigation.

3.2.2. Classification of sensors as Internal Sensors and External Sensors:

Internal Sensors: Used to monitor the internal state of the robot. Examples include position sensors, velocity sensors, acceleration sensors, and force sensors.

External Sensors: Used to gather information about the robot's environment. Examples include contact type sensors (e.g., touch sensors) and non-contact type sensors (e.g., infrared sensors).

Functions of sensors: Sensors provide the robot with data about its environment or its internal state, which is essential for decision-making and control. They enable the robot to interact with its surroundings and adapt its behavior accordingly.

3.3. Actuators

Actuators are the components responsible for the robot's movement and manipulation of objects. Actuators allow robots to actually execute the task they are meant to do. They execute the processed data from the microcontrollers and convert it into motion. It gives mainly two abilities to the robots- Locomotion and Manipulation.

3.3.1. Types of actuators:

Linear actuators: Convert electrical energy into linear motion. Example: solenoids.

Rotary actuators: Convert electrical energy into rotational motion. Example: motors.

Examples: In a robotic arm, linear actuators can be used to extend or retract segments, while rotary actuators can be used to rotate joints or wheels.

3.4. Controller for a Robotic System

Control systems are used in various applications to regulate the behavior of a system. Examples include the regulation of a fan, refrigerator, and air conditioner.



Control systems in robotics: In robotics, control systems are responsible for processing data from sensors, making decisions based on that data, and sending commands to actuators to control the robot's actions.

Differences between manual and automatic control systems: Manual control systems require human intervention, while automatic control systems operate autonomously based on programmed instructions and sensor inputs.

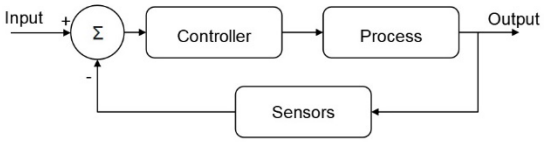


Figure 34: Block Diagram of controller for robotics system.

3.5. Integrating Sensors, Actuators, and Controller in a Robotic System

Let's consider a robotic arm with a revolute joint controlled by a motor. The angular position of the joint is measured by a position sensor. The sensor data is sent to the control system, which processes the data and sends commands to the motor to control the joint's movement. The position sensor provides feedback to the control system, allowing it to make adjustments and maintain the desired position.



4. Visualization, Design and Creation of Components

4.1. Application of Mechanical Block in Robotics:

In the realm of robotics, the mechanical block is crucial for visualizing, designing, and creating individual components. This involves understanding the mechanical properties and functionalities of parts such as actuators, motors, gears, joints, and frames. Tinkercad, a 3D design tool, facilitates the digital modeling of robot components, making it accessible for beginners.

4.1.1. Revolute Joint:

- **Definition:** Allows rotation around a single axis (rotary or hinge joint).
- **Application Example:** Robotic arms in assembly lines for tasks like welding and packaging.

4.1.2. Prismatic Joint:

- **Definition:** Enables linear motion along a single axis (linear or sliding joint).
- **Application Example:** Automated Storage and Retrieval Systems (AS/RS) in warehouses for precise linear movement in item retrieval.

4.1.3. RR Mechanism (Rotary-Rotary):

- **Definition:** Configuration of two revolute joints in series, providing enhanced range of motion.
- **Application Example:** Articulated robotic arms in medical surgery for minimally invasive procedures, offering flexibility and precise movements.

In summary, the mechanical block in robotics plays a vital role in creating components for various applications, from industrial automation to medical surgery, utilizing joints like revolute, prismatic, and mechanisms such as RR for enhanced functionalities.

4.2. Visualization of motion

Visualization of motion refers to the graphical representation of the movement and behavior of a robot or a robotic system.

- **Definition:** It's the graphical representation of a robot's movement and behavior.
- **Tinkercad Usage:** Tinkercad facilitates motion visualization by creating animations of a robot's components. This includes demonstrating how joints and limbs move in response to commands, aiding in understanding mechanical influences on movement.



5. Integrating Robots as a System

Building simple robotic systems, wheeled mobile robot, Single Board Computer coding: Clearly outline the tasks your wheeled mobile robot will perform, whether it's simple navigation, obstacle avoidance, or more complex activities.

1. **Chassis Design:** Construct a sturdy chassis, providing ample space for motors, sensors, and the control system. Material like acrylic, plywood, or metal can be used.
2. **Wheel and Motor Selection:** Choose wheels suitable for the robot's purpose and motors based on weight and desired speed. Options include DC, servo, or stepper motors.
3. **Motor Driver:** Integrate a motor driver circuit to control motor speed and direction, ensuring compatibility with the chosen microcontroller like Arduino.
4. **Power Source:** Utilize a reliable battery pack with matching voltage for the motors and driver. Battery packs are preferred for their lightweight nature.
5. **Sensors:** Select sensors based on the robot's function, such as ultrasonic or infrared sensors for obstacle detection or encoders and gyroscopes for navigation.
6. **Programming:** Code the microcontroller (e.g., Arduino) in languages like C++ or Python for motor control, sensor input, and logical decision-making based on sensor data.
7. **Testing and Troubleshooting:** Rigorously test the robot's movement, responsiveness to sensors, and overall functionality. Troubleshoot any hardware or software issues encountered.
8. **Iteration:** Continuously refine the design, adjusting parameters like chassis structure, motor speeds, or code logic for improved performance.

5.1. Single Board Computer (SBC) Coding for Robotics:

1. **Choosing SBC:** Consider factors like processing power, memory, and compatibility with sensors and actuators when selecting an SBC (e.g., Raspberry Pi for versatility, Arduino for simplicity).
2. **Development Environment:** Set up the development environment with necessary software, tools, and libraries. Python is common for Raspberry Pi, while Arduino uses C/C++.
3. **Writing Code:** Develop code for sensor interaction and actuator control. Understand program structure, data reading from sensors, and actuator control logic.
4. **Debugging and Troubleshooting:** Address common coding issues through debugging techniques, such as print statements and verifying physical connections.

5.2. Building a Mobile Robot with Four Wheels:

1. **SBC Selection:** Choose an SBC based on project requirements, considering factors like processing power, memory, and compatibility.
2. **Motor Control:** Use a motor driver capable of handling four motors (e.g., L298N or L293D). Implement Pulse Width Modulation (PWM) for motor speed control.
3. **Power Source:** Opt for a rechargeable Lithium-ion or Lithium-polymer battery matching voltage requirements for both SBC and motors.
4. **Sensors and Actuators:** Include ultrasonic and infrared sensors for obstacle detection, along with servo motors for specific tasks.



5. **Design and Build:** Design a balanced and stable chassis using material like acrylic or metal. Securely attach components for stability.
6. **Programming:** Code in the preferred language (Python for Raspberry Pi, C/C++ for Arduino) to initialize motor drivers, read sensor data, and implement robot behavior.
7. **Testing and Troubleshooting:** Test the robot's movement, address any unexpected behavior through coding adjustments or hardware checks.
8. **Further Exploration:** Enhance functionality by adding features like computer vision, wireless control, or machine learning for autonomous navigation.

   Encourage experimentation and modification to foster continuous learning and exploration.



Artificial Intelligence

Overview of Syllabus

Note: Key concepts of Class IX need to be revised as a prerequisite.

1. Decision making in Machines/ Computers

(i) Automated versus Autonomous Systems.
Concept of Automated versus Autonomous Systems for Deterministic versus Probabilistic versus.

(ii) Decision Making.
Human versus machine decision making as subjective and objective respectively; An understanding of object classification by humans and computers/machines.

(iii)Machine Learning (ML).
A brief understanding of Machine Learning, the role of data and information. Steps in machine learning. Importance of programming and algorithms in teaching machines/computers in subjective decision making.
Examples such as fruit sorting.

2. Machine Intelligence and Cybersecurity in Computing

(i) Machine Intelligence – Turing Test.
Human intelligence vs Machine Intelligence; role of the Turing test in AI: a brief understanding only; connectivity between human intelligence and machine intelligence.

(ii) Cybersecurity
A basic understanding of security and ethical issues such as the unauthorized use of hardware, theft of software, disputed rights to products, the use of computers to commit fraud, the phenomenon of hacking and data theft, sabotage in the form of viruses, responsibility for the reliability of output, making false claims for computers, and the degradation of work.

3. Components of AI Project Framework

(i) Problem Scoping
Understanding of problem and finding out which factors affect the problem, defining the goal of the project. The 4 Ws: Who, What, Where, Why. The Problem Statement.

(ii) Data Acquisition
Types of Data, Data Features, Data Sources, Training and testing Data and System Maps. Importance of acquiring relevant data from reliable sources.
Sources of Dataset in AI – Kaggle Platform

(iii) Data Exploration
An understanding of different ways of representing data to gather meaningful information: Bar graphs/ histograms, line graphs, scatter plots, pie charts.

(iv) Modelling and Evaluation
Approaches in AI data modelling: Prediction – linear regression. Rule based approach, learning based approach, supervised learning, unsupervised learning (brief understanding of each).



4. Introduction to Data and Programming with Python

(i) Modules and Packages.

Scope and uses of packages, modules and libraries like NumPy, Pandas, SciPy, Matplotlib; implementation using simple programs in python.

An understanding of what libraries are and why we need them in the first place, the syntax of importing libraries.

(ii) Lists and Tuples in Python.

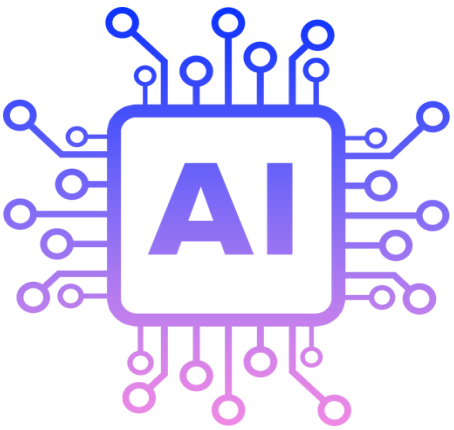
Introduction to lists, creation of list, access elements of a list, list operations (append, insert, extend, sort, search)

Introduction to tuples, creation of tuple, accessing tuples, deleting a tuple, and converting list to tuple and vice-versa.

(iii) Strings

Introduction to strings, creating strings, traversing a string, multiline strings, concatenating strings, escape sequences. String operators, string in-built functions:

int string.len()
string string.upper()
string string.upper()
string string.lower()
boolean string.endswith(value)
boolean string.startswith(value)
int string.find(value)
boolean string.islower()
string string.replace(oldvalue, newvalue)
int string.count(value)
boolean string.isalpha()
boolean string.isdigit()
boolean string.isalnum()
string string.join(iterable)
string string[start:end:step]





1. Decision-Making in Machines

1.1. Automated versus Autonomous Systems

Explore machine decision-making in AI, focusing on the contrast between Automated and Autonomous Systems.

Automated Systems: Follow predefined rules, deterministic, consistent output (e.g., thermostat).

Autonomous Systems: Higher intelligence, adaptability, probabilistic decision-making (e.g., self-driving cars).

Deterministic Decision Making: Certain outcomes based on rules, common in automated systems (e.g., vending machine).

Probabilistic Decision Making: Uncertain outcomes, involves randomness, prevalent in autonomous systems (e.g., self-driving cars).

Understanding these distinctions is crucial for comprehending AI decision-making complexities.

1.2. Decision Making

Introduction: Exploring after machine decision-making, now delve into human vs. machine decision processes. Highlighted the contrast between subjective (human) and objective (machine) decision-making.

(a) *Human Decision Making*: Influenced by emotions and context. Example: Career choices based on personal values and societal expectations.

(b) *Machine Decision Making*: Objective, data-driven, devoid of emotions. Example: Automated email filters using predefined rules to classify spam.

(c) *Object Classification*:

- *Human Object Classification*: Exceptional at rapid recognition based on experience. Example: Easily distinguishing animals in pictures.
- *Machine Object Classification*: Uses machine learning algorithms, relies on labeled data. Example: Smartphone image recognition classifying photos based on learned visual features.

In summary, we've covered the distinctions between human and machine decision-making, emphasizing the contrasting nature of subjective and objective approaches. Additionally, explored how both excel at object classification using unique methods.

1.3. Machine Learning

Machine Learning is a Subset of AI teaching machines to learn and improve without explicit programming, recognizing patterns and making data-driven decisions.

Steps in Machine Learning:

1. Data Collection: Gather relevant data.
2. Data Preprocessing: Clean and prepare data for analysis.
3. Training: Teach the machine using labeled data.
4. Testing and Validation: Evaluate performance on new data.
5. Deployment: Implement ML model in real-world applications.

Programming and Algorithms: Crucial in executing ML algorithms, providing a framework for subjective decision-making.

Practical Example: The steps involved in building a machine-learning model to predict student scores

- **Data Collection**: We use a small dataset of student scores.
- **Data Preprocessing**: In this simple example, the data is already clean.
- **Training**: We train a linear regression model to learn the relationship between assignment/midterm scores and final exam scores.
- **Testing and Validation**: We evaluate the model's performance using mean squared error.
- **Deployment**: We use the trained model to predict the final exam score for a new student based on their assignment and midterm scores.



2. Machine Intelligence and Cybersecurity in Computing

2.1 Machine Intelligence - The Turing Test

Introduction: The Turing Test is a landmark in AI history proposed by Alan Turing. The test assesses a machine's ability to mimic human intelligence convincingly through conversation.

Human Intelligence vs. Machine Intelligence:

Human Intelligence:

- Emotional and Creative.
- Learns from few examples.
- Understands context, appreciates art, and experiences a range of emotions.

Machine Intelligence:

- Data-driven, relies on data and algorithms.
- Consistent and excels at repetitive tasks.

Connectivity Between Human and Machine Intelligence:

- *Assistive Technologies:* Machines aid in various tasks.
- *Cybersecurity:* AI-driven algorithms enhance threat detection and response.

2.2 Cybersecurity

Introduction: Critical in today's interconnected world, Cybersecurity protects computer systems, networks, and data from unauthorized access, theft, or damage.

Security and Ethical Issues in Computing:

(a) Unauthorized Use of Hardware: Risks data breaches.
(b) Theft of Software: Software piracy leads to financial losses.
(c) Disputed Rights to Products: Conflicts over intellectual property.
(d) Computer Fraud: Misusing systems for identity theft and scams.
(e) Hacking and Data Theft: Unauthorized access to steal sensitive data.
(f) Sabotage with Viruses: Malicious software disrupts systems.
(g) Responsibility for Output: Ensuring accurate and reliable results.
(h) False Claims for Computers: Misleading advertising and claims.
(i) Degradation of Work: Over-reliance on computers impacting jobs.

Ethical Considerations in Cybersecurity:

(a) Respecting Privacy: Safeguarding personal data.
(b) Honesty and Transparency: Providing accurate information.
(c) Fair Use of Resources: Ethical use of computing resources.
(d) Cybersecurity Education: Promoting awareness and ethical practices.



3. Components of AI Project Framework

AI projects are complex endeavors that require careful planning and structured execution. The components of an AI project framework provide a structured approach to manage AI initiatives from start to finish.

3.1. Problem Scoping

Problem scoping is the foundational step in any AI project. This phase is essential for defining the overarching goal of the project and setting clear boundaries.

3.1.1. Key Elements of Problem Scoping:

(a) Understanding the Problem**:** Begin by thoroughly comprehending the problem at hand. What are its nuances, complexities, and implications? This phase may involve interviews, research, and data collection to gain insights.

(b) The 4 Ws - Who, What, Where, Why:

- Who: Identify the stakeholders and individuals or entities impacted by the problem. Understanding the people involved is crucial.
- What: Define the problem in precise terms. What are the specific challenges or issues that need to be addressed?
- Where: Consider where the problem is most prevalent or where its impact is most significant. Geographical or contextual factors may play a role.
- Why: Investigate the underlying causes of the problem. Why does it exist, and what are its root causes?

(c) The Problem Statement: Create a concise and clear problem statement that summarizes the essence of the problem, its significance, and the project's objectives. A well-crafted problem statement serves as a guiding beacon for the entire AI project.

3.2. Data Acquisition

3.2.1. Introduction

In the world of Artificial Intelligence (AI), data is the lifeblood that fuels machine learning models. To build effective AI systems, you need to acquire, prepare, and utilize data strategically. This section of our primer explores the crucial component of **Data Acquisition** in AI projects.

3.2.2. Types of Data

Data comes in various forms, each with its unique characteristics. Here are some common types of data:

- **Structured Data:** Highly organized and follows a specific format, such as data in spreadsheets or relational databases.
- **Unstructured Data:** Doesn't have a predefined structure and includes text, images, audio, and video files.
- **Semi-Structured Data:** Falls between structured and unstructured data and includes data with some level of organization but not as rigid as structured data (e.g., JSON files).

3.2.3. Data Features

Data features are the specific characteristics or attributes of your data that you'll use for analysis. For instance, if you're working with a dataset of customer information, features might include age, gender, income, and purchase history.

Data Sources: Data can be sourced from various places, including:

- **Internal Data:** Data generated or collected from within your organization, such as customer records or transaction history.
- **External Data:** Data from sources outside your organization, like public datasets, social media, or government databases.



3.2.4. Training and Testing Data

In AI, it's common to split your data set into two subsets:

- **Training Data:** This subset is used to train your machine learning model. It helps the model learn patterns and relationships within the data.
- **Testing Data:** This subset is used to evaluate your model's performance. It contains data that the model hasn't seen during training and helps assess how well it generalizes to new data.

System Maps:

System maps or data flow diagrams are visual representations of how data moves through your AI system. They illustrate the flow of data from sources through various processing steps to the final output.

Importance of Acquiring Relevant Data:

Acquiring relevant data from reliable sources is critical for several reasons such as:

- **Accuracy:** Reliable data ensures the accuracy of your AI model's predictions or decisions.
- **Generalization:** Relevant data helps your model generalize well to real-world scenarios.
- **Ethics and Bias:** Careful data acquisition helps identify and mitigate biases in the data, ensuring fairness and ethical use of AI.

3.2.5. Sources of Datasets in AI - Kaggle Platform

Kaggle is a popular platform for data scientists and AI practitioners. It offers a vast collection of datasets for various domains and challenges. Here's why Kaggle is valuable:

- **Diverse Datasets:** Kaggle hosts a wide range of datasets, from healthcare to finance, allowing practitioners to explore different domains.
- **Competitions:** Kaggle also hosts machine learning competitions where participants can apply AI techniques to solve real-world problems and win prizes.

3.3. Data Exploration

3.3.1. Introduction

In the journey of working with data, one of the most critical steps is **Data Exploration**. This phase involves digging into your data set to uncover patterns, relationships, and insights that can guide your decision-making. Visualizations play a crucial role in making data more understandable and meaningful.

3.3.2. Different Ways of Representing Data

- **Bar Graphs:** Bar graphs are used to display categorical data, where each category is represented by a separate bar. The height or length of the bar corresponds to the value of that category.
- **Histograms:** Histograms are similar to bar graphs but are primarily used to represent the distribution of continuous data. The x-axis represents intervals or ranges of values, while the y-axis shows the frequency or count of data points falling within each interval.
- **Line Graphs:** Line graphs are ideal for showing trends and changes in data over time. They consist of data points connected by lines, with time usually represented on the x-axis.
- **Scatter Plots:** Scatter plots are used to visualize the relationship between two continuous variables. Each data point is represented as a dot on the graph, and the position of the dot corresponds to the values of the two variables.
- **Pie Charts:** Pie charts are useful for displaying the composition of a whole. They represent parts of a whole as slices of a pie, with each slice proportional to the percentage it represents.

3.3.3. Importance of Data Visualization

Effective data visualization offers several advantages:

- **Clarity:** Visualizations make complex data more understandable and accessible, allowing for quicker comprehension.



- **Pattern Recognition:** Visualizations can reveal patterns and trends in data that might not be apparent in raw numbers.
- **Communication:** Visualizations facilitate the communication of findings and insights to stakeholders or a broader audience.
- **Decision-Making:** Well-crafted visualizations can aid in informed decision-making by presenting data in a clear and actionable manner.

3.4. Modelling and Evaluation

3.4.1. Introduction

In the world of Artificial Intelligence (AI), **data modelling** is a crucial step where we create algorithms and systems that can make predictions, decisions, and discover hidden patterns from data. This section of our primer explores different approaches in AI data modelling, each with its unique characteristics.

Prediction - Linear Regression

Linear regression is a straightforward and widely used approach in AI for making predictions. It is particularly useful when we want to predict a continuous numerical outcome based on one or more input variables. Here's how it works:

- **Objective:** Linear regression aims to find the best-fitting linear relationship between the input variables (independent variables) and the output variable (dependent variable).
- **Representation:** The relationship is typically represented by an equation of the form: Y = aX + b, where "Y" is the predicted value, "X" is the input variable, "a" is the slope, and "b" is the intercept.
- **Usage:** Linear regression is commonly used for tasks like predicting house prices based on features like square footage, number of bedrooms, and location.

3.4.2. Rule-Based Approach

Rule-based systems rely on explicitly defined rules to make decisions or perform tasks. These rules are typically created manually by domain experts or developers. Here's how it works:

- **Objective:** The goal is to create a set of rules that can mimic human decision-making in a specific domain.
- **Representation:** Rules are represented as "if-then" statements. For example, "If the temperature is above 30 degrees Celsius, then turn on the air conditioner."
- **Usage:** Rule-based systems are used in applications like expert systems, where they provide recommendations or make decisions based on predefined rules.

Learning-Based Approach

Learning-based approaches involve training AI models to learn from data and make predictions or decisions. These approaches can be categorized into two main types:

Supervised Learning

- *Objective*: In **supervised learning**, the model is trained on a labeled dataset, where each data point is associated with a known outcome.
- *Representation*: The model learns to map input data to output labels by identifying patterns and relationships in the training data.
- *Usage*: Supervised learning is used for tasks like image classification (e.g., recognizing cats and dogs) and sentiment analysis (e.g., determining if a movie review is positive or negative).

Unsupervised Learning

- *Objective*: In **unsupervised learning**, the model explores unlabeled data to discover hidden patterns, structures, or groupings.
- *Representation*: The model identifies similarities and differences between data points without guidance from labeled examples.
- *Usage*: Unsupervised learning is used for tasks like clustering (e.g., grouping similar customer profiles) and dimensionality reduction (e.g., reducing the complexity of data).



4. Introduction to Data and Programming with Python

4.1. Modules and Packages

In the world of programming and artificial intelligence, Python is a versatile and powerful language. To harness its full potential, we often rely on modules and packages. These are collections of pre-written code that extend Python's capabilities, making complex tasks simpler. Let's explore them:

Modules:

- A module is a single Python file containing reusable code.
- You can create your own modules or use built-in ones.

Packages:

- A package is a collection of related Python modules organized in directories.
- Some popular packages in AI include NumPy, Pandas, SciPy, and Matplotlib.

NumPy (Numerical Python):

- NumPy is a fundamental package for scientific computing in Python.
- NumPy is crucial for tasks like data manipulation and linear algebra.

Pandas:

- Pandas is a library used for data manipulation and analysis.
- Pandas simplifies working with structured data like spreadsheets or SQL tables.

SciPy (Scientific Python):

- SciPy builds on NumPy and adds additional functionality.
- It includes modules for optimization, integration, interpolation, eigenvalue problems, and more.
- SciPy is vital for scientific and technical computing.

Matplotlib:

- Matplotlib is a popular data visualization library.
- Visualizing data is essential for gaining insights and making informed decisions.

Example: Write a python program to create a straight line

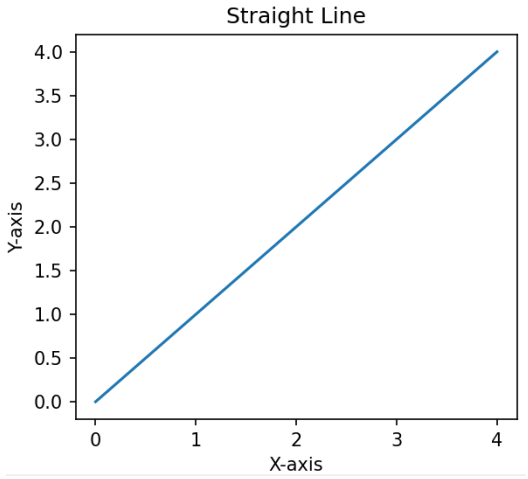
```
import matplotlib.pyplot as plt

# Define data for the straight line
x_values = [0, 1, 2, 3, 4]  # x values
y_values = [0, 1, 2, 3, 4]  # y values (in this case, just the same as x for a straight line)

# Plot the straight line
plt.plot(x_values, y_values)

# Add labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Straight Line')

# Show the plot
plt.show()
```



Importing Libraries:

- To use a library or module in Python, you must first import it.
- The **import** statement is used to bring external code into your program.
- For example, to import NumPy, you'd use: **import numpy as np** (this assigns an alias 'np' for convenience).

Why Do We Need Libraries? Libraries are essential because they:

- *Save time*: They provide pre-built functions, so you don't have to reinvent the wheel.
- *Improve efficiency*: Libraries are optimized for performance.



- *Enhance capabilities:* They extend Python's functionality for specific tasks.
- *Promote code reusability:* You can use libraries across multiple projects.

Implementation with Python: Example of using NumPy to calculate the mean of an array:

```
import numpy as np

# Creating an array with some sample data
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Calculating the mean of the array
mean_value = np.mean(data)

# Printing the mean value
print("The mean of the array is:", mean_value)
```

Output

```
The mean of the array is: 5.5
```

Implementation with Python: Example of using Pandas to convert

```
import pandas as pd

# Create a DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22]
}

df = pd.DataFrame(data)

# Display the DataFrame
print("DataFrame:")
print(df)

# Calculate the average age
average_age = df['Age'].mean()
print("\nAverage Age:", average_age)
```

Output :

```
DataFrame:
      Name  Age
0    Alice   24
1      Bob   27
2  Charlie   22

Average Age: 24.333333333333332
```

4.2. Lists and Tuples in Python

Lists:

- **Introduction to Lists:** Lists are one of the most versatile and commonly used data structures in Python. They are sequences of elements, where each element can be of any data type (e.g., integers, strings, floats).

Creation of a List:

- You can create a list by enclosing elements in square brackets **[]**. For example:
  Python code
  my_list = [1, 2, 3, 4, 5]

Accessing Elements of a List:

- Lists are indexed, and you can access individual elements using square brackets and the index. Indexing starts from 0.
  Python code
  first_element = my_list[0] # Accesses the first element (1)

List Operations:

- Lists support various operations for manipulation:
- **Append:** Adds an element to the end of the list.
  Python code
  my_list.append(6) # Adds 6 to the end of the list
- **Insert:** Inserts an element at a specific position.
  Python code
  my_list.insert(2, 7) # Inserts 7 at index 2
- **Extend:** Appends elements from another list to the end.
  Python code
  another_list = [8, 9]
  my_list.extend(another_list) # Appends [8, 9] to my_list
- **Sort:** Sorts the list in ascending order.
  Python code
  my_list.sort() # Sorts the list in-place
- **Search:** You can search for elements in a list using various methods. For example, to check if an element exists:



Python code
If 4 in my_list: print ("4 is in the list")

Tuples:

Introduction to Tuples: Tuples are similar to lists but are immutable, meaning their elements cannot be changed after creation. They are often used for data that should not be modified.

Creation of a Tuple:

- You can create a tuple by enclosing elements in parentheses **()**. For example:
- Python Copy code
  my_tuple = (1, 2, 3)

Accessing Tuples:

- Tuples are indexed just like lists, and you can access elements using square brackets and the index.

Deleting a Tuple:

- Tuples are immutable, so you cannot delete individual elements. However, you can delete the entire tuple using the **del** statement:
  Python code
  del my_tuple

Converting List to Tuple and Vice Versa:

- You can convert a list to a tuple using the **tuple ()** constructor:
  Python code
  my_list = [1, 2, 3] my_tuple = tuple(my_list) # Converts the list to a tuple

- To convert a tuple to a list, you can use the **list ()** constructor:
  Python code
  my_tuple = (4, 5, 6) my_list = list(my_tuple) # Converts the tuple to a list

4.3. Strings

Introduction to Strings:

- **Strings** are sequences of characters, such as letters, numbers, and symbols.
- They are enclosed in either single quotes (‘’) or double quotes (“”).

Creating Strings:

- To create a string, simply enclose text in quotes. For example:
  Python code
  my_string = “Hello, World!”

Traversing a String:

- You can traverse a string character by character using a loop. For example, using a **for** loop:
  Python code
  for char in my_string: print(char)

Multiline Strings:

- Multiline strings are created using triple quotes ("" or """), allowing you to include line breaks.
  Python code
  multiline_string = '"This is a multiline string."'

Concatenating Strings:

- You can concatenate (combine) strings using the + operator:
  Python code
  greeting = "Hello, " name = "Alice" message = greeting + name

Escape Sequences:

- Escape sequences are special characters preceded by a backslash (\). Common escape sequences include:
- **'\n'** for a newline.
  - **'\t'** for a tab.
  - '\"' for a single quote.
  - '\"' for a double quote.
  - '\\' for a backslash.

String Operators:

- Python supports various string operators, such as + for concatenation and * for repetition.

String Built-in Functions: Few Examples

- **len():** Returns the length of the string.
- **upper():** Converts the string to uppercase.
- **lower():** Converts the string to lowercase.
- **endswith(value):** Checks if the string ends with the specified value.
- **startswith(value):** Checks if the string starts with the specified value.
- **find(value):** Returns the index of the first occurrence of the specified value (or -1 if not found).



Example:

Python code

```
# Creating strings
single_quote_string = 'Hello'
double_quote_string = "World"
multiline_string = """This is
a multiline
string"""
# Traversing a string
for char in single_quote_string:
    print(char)
# Multiline string
print(multiline_string)

# Concatenating strings
greeting = single_quote_string + ' ' +
double_quote_string
print(greeting)  # Output: Hello World

# Escape sequences
escaped_string = "He said, \"Hello!\""
print(escaped_string)  # Output: He said,
"Hello!"

# String operators
repeated_string = "Ha" * 3
print(repeated_string)  # Output: HaHaHa

# In-built string functions
my_string = "Hello, World!"
print(len(my_string))  # Output: 13
print(my_string.upper())  # Output: HELLO,
WORLD!
print(my_string.lower())  # Output: hello,
world!
print(my_string.endswith("World!"))  #
Output: True
print(my_string.startswith("Hello"))  #
Output: True
print(my_string.find("World"))  # Output: 7
print(my_string.islower())  # Output: False
print(my_string.replace("World",
"Python"))  # Output: Hello, Python!
print(my_string.count("o"))  # Output: 2
print(my_string.isalpha())  # Output: False
print("12345".isdigit())  # Output: True
print("Hello123".isalnum())  # Output: True
my_list = ['Hello', 'World']
print(", ".join(my_list))  # Output: Hello,
World
print(my_string[0:5])  # Output: Hello
print(my_string[::2])  # Output: Hlo ol!
```

These string operations and built-in functions are fundamental for text processing, data manipulation, and working with strings in Python. This enables to perform various tasks efficiently and effectively when dealing with textual data in various AI projects.



V. Suggestive Interactive Activities and Assignments for Classes IX & X



ROBOTICS

Class IX

Pre-Activity Preparation

- Team Formation: Form teams ensuring a mix of abilities and interests. Consider an odd number of members in each team to avoid ties during decision-making.
- Schedule Planning: Allocate time for each activity, including setup and wrap-up. Ensure a balance between active and reflective phases.

Activity 1

Robot Classification Activity

Objective:

To introduce beginners to the exciting world of Robotics through the discussion among teams.

Material Needed

- Computers with internet access.
- Different images of robots- e.g Drones, Mobile robots, manipulators

Procedure:

- ***Introduction (5 minutes):***
  - Briefly explain the classification of robot.
  - Emphasize the convenience of image representation of robots.
- ***Hands-On Exercise (15 minutes)*:**
  - Instruct students to write about description of robots.
  - Assist in writing about robots – knowhow about robots and its applications.
- ***Interactive Quiz (10 minutes):***
  - Conduct a quiz within Teams with questions related to robots and its applications
- ***Conclusion (5 minutes):***
  - Summarize the learning outcomes and highlight the description of robots.

This interactive session leverages to introduce robots and its applications.



Activity 2

Designing a Sustainable Robotic Arm

Objective:

To introduce participants to the design and construction of a robotic arm using sustainable material, fostering awareness of environmental considerations in engineering projects.

Material Needed

- Sustainable material such as recycled cardboard, bamboo, or biodegradable plastics.
- Cutting tools (scissors, box cutters) for working with material.
- Glue or tape for assembly.
- Computers with internet access (for research and design).

Procedure:

- ***Introduction (5 minutes):***
  - Introduce the concept of designing a robotic arm using sustainable material, emphasizing the importance of environmental consciousness in engineering.
  - Discuss the significance of using recycled or biodegradable material in reducing environmental impact.
- ***Design Demonstration (10 minutes):***
  - Showcase examples of robotic arms made from sustainable material, highlighting their functionality and feasibility.
  - Discuss design considerations such as durability, flexibility, and eco-friendliness when selecting material.
  - Guide how to sketch initial design concepts and consider factors like size, range of motion, and load capacity.
- ***Hands-On Exercise (15 minutes):***
  - Instruct participants to brainstorm and sketch their robotic arm designs using sustainable material.
  - Provide sustainable material and cutting tools for participants to begin prototyping their designs.
  - Offer guidance and support as participants assemble their robotic arms, encouraging creativity and experimentation with different material and configurations.
- ***Interactive Quiz (10 minutes):***
  - Conduct a quiz on sustainability principles in engineering, focusing on topics like material selection, recycling, and environmental impact mitigation.
  - Use interactive tools to engage participants and reinforce key concepts related to sustainability.



- ***Material Exploration (10 minutes):***
  - Explore various sustainable material suitable for robotic arm construction through research and discussion.
  - Discuss the advantages and limitations of each material in terms of strength, weight, flexibility, and environmental impact.
  - Encourage participants to consider the lifecycle of material, from sourcing to disposal, in their design decisions.
- ***Conclusion (5 minutes):***
  - Summarize the importance of sustainable engineering practices in reducing environmental impact.
  - Showcase the robotic arms created by participants, highlighting their innovative designs and use of sustainable material.
  - Facilitate a discussion on the challenges and opportunities in designing with sustainability in mind, encouraging future exploration and collaboration in eco-friendly engineering projects.

    Note: This interactive session engages participants in the design and construction of a robotic arm using sustainable material, promoting environmental awareness and creative problem-solving in engineering.



Activity 3

LED Blinking with Tinkercad!

Objective:

To introduce beginners to the fundamental concept of programming an LED to blink using Arduino and Tinkercad's simulation environment.

Material Needed:

- Computers with internet access.
- Tinkercad accounts for each participant.

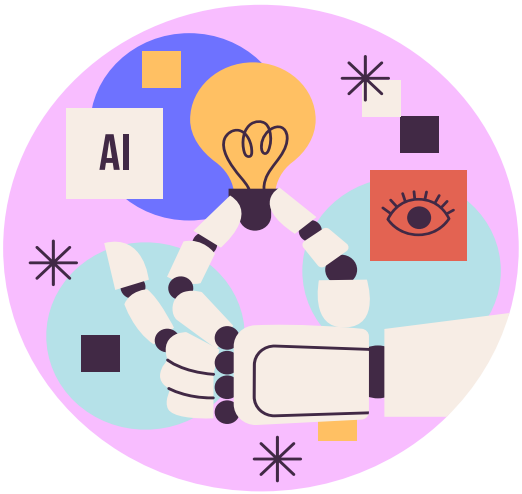
Procedure:

- ***Introduction (5 minutes):***
  - Briefly explain that Arduino can be simulated online using Tinkercad.
  - Emphasize the convenience of virtual experimentation before physical setups, focusing on the simplicity of LED blinking as a starting point.
- ***Code Demonstration (10 minutes):***
  - Showcase a simple Arduino program using Tinkercad's virtual Arduino and components, specifically demonstrating LED blinking.
  - Explain the basic structure of an Arduino program and how to use Tinkercad's code editor to control the LED.
- ***Hands-On Exercise (15 minutes):***
  - Instruct students to log in to Tinkercad and open the Arduino simulator.
  - Guide them through the hands-on exercise of creating a virtual circuit with an LED and a resistor.
  - Assist in writing and simulating a program to make the LED blink at a desired rate.
  - Encourage creative modifications to the simulation, such as changing the blink pattern or adding additional LEDs.
- ***Interactive Quiz (10 minutes):***
  - Conduct a quiz within Tinkercad with questions related to LED blinking, Arduino syntax, and basic circuit concepts.
  - Utilize Tinkercad's platform for real-time quiz participation and instant feedback.
- ***Code Exploration (10 minutes):***
  - Explore Tinkercad's code snippets library for various LED blinking patterns and Arduino applications.
  - Demonstrate how Tinkercad allows for simulation and testing before physical implementation, encouraging students to experiment with different code snippets.



- ***Conclusion (5 minutes):***
  - Summarize the learning outcomes, highlighting the understanding gained about LED blinking and basic Arduino programming.
  - Emphasize the advantages of Tinkercad for virtual Arduino projects, including its ease of use and accessibility.
  - Encourage students to explore further projects on Tinkercad independently, fostering their curiosity and creativity in electronics and programming.

Note: This interactive session leverages Tinkercad to introduce LED blinking with Arduino, providing a hands-on experience in programming and circuit building in a virtual environment.



Activity 4

Exploring the Laws of Robotics

Objective:

To facilitate a discussion on Asimov's Three Laws of Robotics and their ethical implications, encouraging critical thinking and reflection on the role of robotics in society.

Material Needed

- Presentation slides summarizing Asimov's Three Laws of Robotics.
- Ethical scenarios related to robotics and artificial intelligence.

Procedure:

- ***Introduction (5 minutes):***
  - Provide a brief overview of the session's objective to discuss Asimov's Three Laws of Robotics and their ethical implications.
  - Highlight the importance of considering ethical principles in the development and implementation of robotics technology.
- ***Discussion Start (10 minutes):***
  - Present a summary of Asimov's Three Laws of Robotics, explaining each law and its significance.
  - Invite participants to share their initial thoughts and interpretations of the laws.
- ***Group Discussion (20 minutes):***
  - Pose ethical scenarios and questions related to the laws, encouraging participants to consider various perspectives and potential outcomes.
  - Facilitate a discussion that explores different viewpoints and encourages critical thinking about the ethical dilemmas presented.
  - Encourage participants to draw connections between the laws and real-world examples of robotics and artificial intelligence.
- ***Debrief (10 minutes):***
  - Summarize the key points raised during the discussion, highlighting any recurring themes or areas of disagreement.
  - Reflect on how Asimov's Three Laws of Robotics impact the development and implementation of robotics technology in the real world.
  - Encourage participants to consider the complexities of applying ethical principles in robotics and to continue exploring these topics beyond the session.

Note: This interactive discussion provides participants with an opportunity to explore the ethical implications of Asimov's Three Laws of Robotics and engage in critical dialogue about the role of robotics in society.



Activity 5

Traffic Light Simulation with Tinkercad!

Objective:

To familiarize beginners with the concept of simulating a traffic light using LEDs and Arduino in Tinkercad's virtual environment.

Material Needed

- Computers with internet access.
- Tinkercad accounts for each participant.

Procedure:

- ***Introduction (5 minutes):***
  - Provide a brief overview of the session's objective to simulate a traffic light using LEDs and Arduino in Tinkercad.
  - Emphasize the relevance of understanding traffic light systems in the context of electronics and programming.
- ***Code Demonstration (10 minutes):***
  - Showcase a simple Arduino program using Tinkercad's virtual Arduino and components to control LEDs as traffic lights.
  - Explain the basic structure of the Arduino program and how to use Tinkercad's code editor to implement the traffic light sequence.
- ***Hands-On Exercise (15 minutes):***
  - Instruct students to log in to Tinkercad and open the Arduino simulator.
  - Guide them through the hands-on exercise of creating a virtual traffic light system with LEDs and resistors.
  - Assist in writing and simulating a program to control the traffic light sequence (red, yellow, green).
  - Encourage creative modifications to the simulation, such as adjusting the timing of the light sequence or adding pedestrian crossing signals.
- ***Interactive Quiz (10 minutes):***
  - Conduct a quiz within Tinkercad with questions related to traffic light systems, Arduino syntax, and circuit concepts.
  - Utilize Tinkercad's platform for real-time quiz participation and immediate feedback.



- ***Code Exploration (10 minutes):***
  - Explore Tinkercad's code snippets library for various traffic light patterns and Arduino applications.
  - Demonstrate how Tinkercad allows for simulation and testing before physical implementation, encouraging students to experiment with different code snippets for traffic light control.
- ***Conclusion (5 minutes):***
  - Summarize the learning outcomes, highlighting the understanding gained about simulating traffic lights with LEDs and Arduino.
  - Emphasize the advantages of Tinkercad for virtual electronics projects, including its ease of use and accessibility.
  - Encourage students to explore further projects on Tinkercad independently, fostering their curiosity and creativity in electronics and programming.

Note: This interactive session leverages Tinkercad to simulate a traffic light system using LEDs and Arduino, providing a hands-on experience in programming and circuit building in a virtual environment.



ARTIFICIAL INTELLIGENCE

Activity 1

Hello, Python!

Objective:

To successfully write and execute a Python program that displays the message "Hello, Python!" on the screen, demonstrating basic understanding of syntax, programming principles, and the output process in Python.

Material Needed

- Computers with Python installed (IDLE or Jupyter Notebook).
- Python code snippets for demonstration.

Procedure:

- ***Introduction (5 minutes):***
  - Begin by explaining that Python is a versatile programming language used in many fields.
  - Emphasize its suitability for beginners due to its readability and simplicity.
- ***Code Demonstration (10 minutes):***
  - Showcase a simple Python program, such as printing "Hello, Python!" on the screen.
  - Explain the basic structure of a Python program, including comments and the use of the print () function.
- ***Hands-On Exercise (15 minutes):***
  - Instruct students to open a Python environment (IDLE or Jupyter Notebook) on their computers.
  - Guide them through writing and running their first Python program to print "Hello, Python!" on the screen.
- ***Interactive Quiz (10 minutes):***
  - Conduct a brief quiz with multiple-choice questions to test their understanding of Pythons uses and advantages.
  - Display snippets of Python code related to various applications (e.g., a simple web page using Flask, a data analysis snippet).
  - Explain how Python's versatility allows it to be used in these different scenarios.



Activity 2

Design Your Own AI Chatbot

Objective:

Design your own AI Chatbot to create a friendly and helpful virtual friend. This activity lets you step into the world of AI in a fun and creative way. As you design and practice conversations, you will get a taste of how AI can interact with people and provide useful information or entertainment. Enjoy the process of creating your AI Chatbot!

Material Needed

Pen and paper

Procedure:

- ***Imagine Your Chatbot's Role*:** Think about what your Chatbot will do. Will it answer questions, give fun facts, or tell jokes?
- ***Give Your Chatbot a Personality*:** Decide if your Chatbot will be serious, funny, friendly, or something else.
- ***Brainstorm Responses*:** Write down different things your Chatbot might say in different situations. For example:
  If someone says "Hello," how will your Chatbot reply?
  If someone asks for a joke, what kind of joke will your Chatbot share?
  Draw the Conversation: Use your responses to create a simple conversation.
  Draw speech bubbles with what the user might say and what your Chatbot might reply.
- ***Practice with a Friend*:** Role-play with a friend, taking turns being the user and the Chatbot. Use your conversation drawing as a guide.
- ***Feedback and Improvement*:** After practicing, ask your friend for feedback. What did they like? What could be better? Make adjustments based on their input.

Assignments:

- Create a presentation highlighting the benefits of using AI in various sectors such as healthcare, finance, and entertainment.
- Write a reflective essay on the ethical considerations in AI, focusing on bias, fairness, and accountability.



Activity 3

Logic Gates and Binary Logic Exploration

Objective:

Introduce students to binary logic and logic gates through hands-on exploration to understand digital circuit fundamentals.

Material Needed

- A whiteboard or flip chart with markers.
- A projector or whiteboard to display logic gate diagrams.
- A set of logic gate cards (AND, OR, NOT) printed on paper.
- Small group or individual worksheets.

Procedure:

- ***Introduction to Logic Gates (15 minutes):***
  - Begin by introducing students to the concept of logic gates and their role in early computing.
  - Explain the three fundamental logic gates: AND, OR, and NOT. Use simple examples and diagrams to illustrate their functions.
  - Discuss how these gates can be combined to perform more complex logical operations.
- ***Logic Gate Demonstrations (15 minutes)*:**
  - Use a projector or whiteboard to display logic gate diagrams and truth tables.
  - Walk through examples of logic gate operations with the class. For instance, demonstrate how an AND gate outputs a '1' only when both inputs are '1'.
  - Encourage students to ask questions and participate in discussions about these operations.
- ***Logic Gate Card Game (20 minutes)*:**
  - Divide the class into small groups or pairs.
  - Distribute logic gate cards to each group. Each card should have a logic gate symbol (AND, OR, or NOT) on one side and a description of its function on the other side.
  - In a fun card game format, students take turns drawing cards and forming logical expressions using the gates. For example, they might create expressions like "A AND B," "NOT C," etc.
  - Encourage them to discuss the outcomes of these expressions and how different combinations of gates affect the results.
- ***Worksheet Challenge (15 minutes)*:**
  - Provide each group or student with a worksheet containing a series of logic gate problems.
  - These problems can involve creating logical expressions, solving truth tables, or simplifying Boolean expressions.
  - Students work individually or in groups to solve the problems.
- ***Group Presentations (10 minutes)*:**
  - Invite each group to present one interesting logic gate problem they encountered and how they solved it.
  - Encourage discussions and explanations and allow the class to ask questions.
- ***Reflection (5 minutes)*:**
  - Conclude the activity with a brief reflection on the significance of logic gates in early computing and their relevance to AI and modern computing.
  - Highlight how logical operations are at the core of decision-making in AI and programming.



Activity 4

Data Detective and Variable Explorer

Objective:

Familiarize students with data analysis techniques, exploring variables, and interpreting data to cultivate critical thinking skills in data interpretation.

Material Needed

- Whiteboard or flip chart with markers.
- Sticky notes.
- Example Python code snippets.

Procedure:

- ***Introduction (5 minutes)*:**
  - Explain the significance of data types and variables in Python programming.
  - Emphasize the importance of selecting the right data type for specific use cases.
- ***Data Detective Activity (15 minutes)*:**
  - Present students with various scenarios (e.g., counting apples, measuring temperature, storing a list of student names).
  - Ask them to identify which data type (numeric, Boolean, sequence, set, or dictionary) would be most appropriate for each scenario.
  - Use sticky notes to label the scenarios with the chosen data types.
- ***Variable Explorer (15 minutes)*:**
  - Introduce the concept of variables by showing how to declare and assign values to them.
  - Provide students with example code snippets containing variables and their assignments.
  - Ask students to identify the data type of each variable and explain their choices.
- ***Code Structure and Indentation (10 minutes)*:**
  - Explain the importance of proper indentation in Python code.
  - Display example code with different levels of indentation to demonstrate code blocks (e.g., loops or conditionals).
  - Discuss how indentation helps in code readability and structure.
- ***Interactive Discussion (10 minutes)*:**
  - Encourage students to share their chosen data types for the scenarios and the reasons behind their choices.
  - Discuss the impact of choosing the wrong data type on program functionality.
  - Highlight the significance of indentation in Python's program structure.



Activity 5

Operator Explorer and Statement Builder

Objective:

Develop students' understanding of programming operators and statement construction through interactive exploration, fostering proficiency in basic programming concepts.

Material Needed

- Whiteboard or flip chart with markers.
- Sticky notes.
- Example Python code snippets.

Introduction:

- ***Introduction (5 minutes):***
  - Explain the concept of operators and their role in Python programming.
  - Mention the types of operators students will explore and how operators are used in real-world programming.
- ***Operator Explorer (15 minutes):***
  - Present students with various data types (numbers, Boolean values, sequences) on sticky notes.
  - Ask students to use these data types and apply different operators to perform operations.
  - For example, they can add two numbers, combine Boolean values with logical operators, or check for membership in a sequence.
- ***Statement Builder (15 minutes):***
  - Explain the concept of statements in Python.
  - Provide students with example code snippets containing variables, operators, and statements.
  - Ask them to create statements that perform specific tasks, such as calculating the average of numbers, checking if a number is even, or iterating through a list.
- ***Code Presentation (10 minutes):***
  - Invite students to present their created statements to the class.
  - Encourage them to explain the purpose of their statements and how operators are used within them.
- **Interactive Discussion (10 minutes):**
  - Engage in a discussion about the importance of operators and statements in Python programming.
  - Highlight real-world examples where these concepts are used in applications.



Class X
ROBOTICS

Pre-Activity Preparation

- Team Formation: Form teams ensuring a mix of abilities and interests. Consider an odd number of members in each team to avoid ties during decision-making.
- Schedule Planning: Allocate time for each activity, including setup and wrap-up. Ensure a balance between active and reflective phases.

Activity 1
Familiarization with IR Sensor Simulation with Tinkercad!

Objective:

To demonstrate the functionality and applications of an IR sensor in detecting objects in Tinkercad's virtual environment.

Material Needed

- Computer with internet access.
- Tinkercad account.

Procedure:

- ***Introduction (5 minutes):***
  - Provide an overview of the experiment's objective to understand how an IR sensor detects objects.
  - Briefly explain the principle of operation of IR sensors in emitting and receiving infrared light.
- ***Setup (5 minutes):***
  - Log in to Tinkercad and open a new circuit design project.
  - Search and add an IR sensor component to the workspace.
- ***Circuit Assembly (10 minutes):***
  - Connect the IR sensor to the Arduino board using jumper wires, ensuring proper wiring according to the sensor's pinout.
  - Double-check the connections to ensure there are no loose connections or errors.
- ***Programming (15 minutes):***
  - Open the Tinkercad code editor and write a simple Arduino program to read the sensor data.
  - Program the Arduino to detect the presence of an object using the IR sensor and output the result to the serial monitor.
- ***Testing (10 minutes):***
  - Simulate the circuit in Tinkercad to observe the behavior of the IR sensor.
  - Place objects of varying material and sizes in front of the sensor to see how it reacts to different obstacles.
  - Observe the changes in sensor readings displayed in the serial monitor as objects are moved closer or farther away from the sensor.



- ***Data Analysis (10 minutes):***
  - Analyze the sensor readings obtained during testing to understand how the IR sensor detects objects.
  - Discuss factors that affect the sensor's performance, such as distance, reflectivity of the object, and ambient light interference.
- ***Conclusion (5 minutes):***
  - Summarize the key findings of the experiment, emphasizing the role of IR sensors in object detection applications.
  - Discuss potential real-world applications of IR sensors, such as proximity sensing, obstacle detection in robotics, and motion detection in security systems.
- ***Exploration (5 minutes):***
  - Encourage participants to explore additional experiments or projects involving IR sensors in Tinkercad or real-world applications.
  - Provide resources or suggestions for expanding their understanding of sensor technologies and their applications.

Note: This experiment provides a hands-on opportunity for participants to learn about IR sensors and their applications through practical experimentation in Tinkercad's virtual environment.



Activity 2

Obstacle Avoidance Robot Simulation with Tinkercad!

Objective:

To introduce beginners to the concept of simulating an obstacle avoidance robot using sensors, motors, and Arduino in Tinkercad's virtual environment.

Material Needed

- Computers with internet access.
- Tinkercad accounts for each participant.

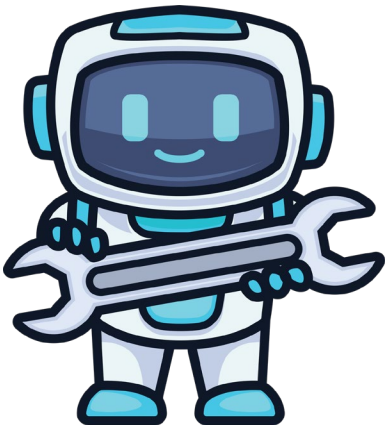
Procedure:

- ***Introduction (5 minutes):***
  - Provide a brief overview of the session's objective to simulate an obstacle avoidance robot using sensors, motors, and Arduino in Tinkercad.
  - Emphasize the importance of understanding sensor-based robotics concepts like obstacle detection and avoidance.
- ***Code Demonstration (10 minutes):***
  - Showcase a simple Arduino program using Tinkercad's virtual Arduino and components to control motors and read sensor data for obstacle detection.
  - Explain the basic structure of the Arduino program and how to use Tinkercad's code editor to implement the obstacle avoidance algorithm.
- ***Hands-On Exercise (15 minutes):***
  - Instruct students to log in to Tinkercad and open the Arduino simulator.
  - Guide them through the hands-on exercise of creating a virtual obstacle avoidance robot with sensors, motors, and Arduino.
  - Assist in writing and simulating a program to control the robot's movement and avoid obstacles based on sensor readings.
  - Encourage creative modifications to the simulation, such as adjusting sensor placement or adding additional behaviors for the robot.
- ***Interactive Quiz (10 minutes):***
  - Conduct a quiz within Tinkercad with questions related to robotics concepts, Arduino syntax, and sensor principles.
  - Utilize the Tinkercad platform for real-time quiz participation and immediate feedback.
- ***Code Exploration (10 minutes):***
  - Explore Tinkercad code snippets library for various robotic behaviors and Arduino applications related to sensor-based obstacle avoidance.
  - Demonstrate how Tinkercad allows for simulation and testing before physical implementation, encouraging students to experiment with different code snippets for robot control.



- ***Conclusion (5 minutes):***
  - Summarize the learning outcomes, highlighting the understanding gained about simulating a sensor-based obstacle avoidance robot using Tinkercad.
  - Emphasize the advantages of Tinkercad for virtual robotics projects, including its ease of use and accessibility.
  - Encourage students to explore further projects on Tinkercad independently, fostering their curiosity and creativity in robotics and programming.

Note: This interactive session leverages Tinkercad to simulate a sensor-based obstacle avoidance robot, providing a hands-on experience in programming and robotics in a virtual environment.



Activity 3

Line Following Olympics

Objective:

Introduce students to basic robotics principles like movement, sensors, and programming through a competitive and engaging line-following challenge.

Material Needed

- Microbit or similar microcontroller boards (one per team)
- Line following sensors (infrared or reflectance)
- Black electrical tape to create various line tracks
- Cardboard or foam board for the playing field
- Markers or tape for obstacles
- Stopwatches or timers
- Scoreboard (optional)

Procedure:

- Divide students into teams of 2-3.
- ***Briefly explain the challenge*:** Design and program robots to autonomously follow black lines on a course, competing for the fastest completion time.
- ***Introduce key concepts*:** Explain how line following sensors work, basic programming logic (if-else statements, loops), and motor control.
- ***Planning and design*:** Teams brainstorm robot designs, considering sensor placement, motor configuration, and program flow. Encourage creativity and problem-solving.
- ***Building phase*:** Teams assemble their robots using Microbits, sensors, and any additional material allowed. Guide them through the assembly process.
- ***Programming phase*:** Students write code for their robots using the Microbit platform or similar software. The code should enable the robot to detect the line, adjust its direction accordingly, and follow the course until reaching the finish line.
- ***Calibration and testing*:** Teams test their robots on practice tracks, fine-tuning their programs and sensor sensitivity for optimal line following.
- ***The Olympics!*:** Each team competes on the official course, attempting the fastest run with their robot. Time each attempt and update the scoreboard.
- ***Evaluation and discussion*:** Teams showcase their robots and explain their design choices and programming strategies. Discuss successes, challenges, and potential improvements. Encourage collaboration and peer learning.

Optional enhancements:

- Introduce different track levels with varying difficulty (sharp turns, intersections, etc.).
- Allow teams to “mod” their robots with additional features (buzzers for finish line, speed variations).
- Organize elimination rounds and a final championship for added excitement.
- Encourage research on real-world line following robots and their applications.

Note: This activity leverages friendly competition to engage students in learning core robotics concepts. It promotes teamwork, critical thinking, and the thrill of optimizing their creations for success.



Activity 4

Controlling a Servo Motor in Tinkercad!

Objective:

To understand the principles of servo motor control and practice implementing servo motor control in Tinkercad's virtual environment.

Material Needed

- Computer with internet access.
- Tinkercad account.

Procedure:

- ***Introduction (5 minutes):***
  - Provide an overview of the experiment's objective: to learn how to control a servo motor using Arduino in Tinkercad.
  - Briefly explain the functionality of servo motors and their applications in robotics and automation.
- ***Tinkercad Setup (5 minutes):***
  - Log in to Tinkercad and open a new circuit design project.
  - Search and add an Arduino board and a servo motor component to the workspace.
- ***Circuit Assembly (10 minutes):***
  - Connect the servo motor to the Arduino board using jumper wires, ensuring proper wiring according to the servo motor's pinout.
  - Double-check the connections to ensure there are no loose connections or errors.
- ***Programming (15 minutes):***
  - Open the Tinkercad code editor and write a simple Arduino program to control the servo motor.
  - Program the Arduino to rotate the servo motor to specific angles at regular intervals, such as 0°, 90°, and 180°.
  - Include delays in the code to control the speed of the servo motor's movement.
- ***Testing (10 minutes):***
  - Simulate the circuit in Tinkercad to observe the behavior of the servo motor.
  - Observe the servo motor's movement as it rotates to the specified angles according to the programmed instructions.
  - Adjust the code as necessary to fine-tune the servo motor's movement and behavior.
- ***Data Analysis (10 minutes):***
  - Analyze the servo motor's performance during testing, including its accuracy in reaching the specified angles and the smoothness of its movement.
  - Discuss factors that may affect the servo motor's performance, such as power supply voltage, mechanical limitations, and programming errors.
- ***Conclusion (5 minutes):***
  - Summarize the key findings of the experiment, highlighting the understanding gained about controlling a servo motor using Arduino in Tinkercad.
  - Discuss potential real-world applications of servo motors and their importance in robotics, automation, and other electromechanical systems.

Note: This experiment provides a hands-on opportunity for participants to learn about servo motor control and its applications through practical experimentation in Tinkercad's virtual environment. Encourage participants to explore further projects and applications involving servo motors to deepen their understanding of this versatile component.



Activity 5

Obstacle Avoidance Robot

Objective:

To design and build a robot capable of autonomously navigating its environment while avoiding obstacles using sensors and actuators.

Material Needed

- Robot chassis with motors and wheels
- Arduino microcontroller board or any other board
- Motor driver module
- Ultrasonic distance sensor (e.g., HC-SR04)
- Jumper wires
- Breadboard
- Battery pack or power source
- Optional: chassis mounting hardware, additional sensors for enhanced obstacle detection

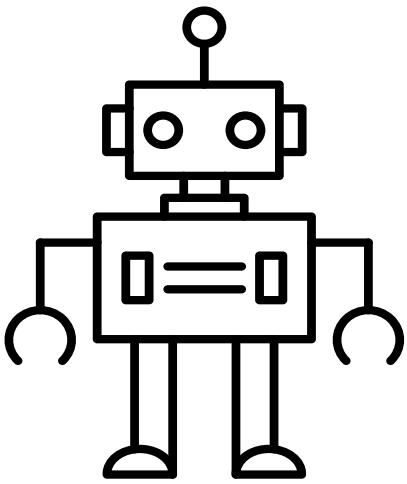
Procedure:

- ***Planning (10 minutes):***
  - Discuss the project objectives, including designing a robot that can navigate autonomously and avoid obstacles.
  - Review the components and their functions, emphasizing the role of sensors and actuators in obstacle avoidance.
- ***Circuit Assembly (20 minutes):***
  - Mount the motors and wheels onto the robot chassis according to the manufacturer's instructions.
  - Connect the motors to the motor driver module and the Arduino board.
  - Wire the ultrasonic distance sensor to the Arduino board, ensuring proper connections for power, ground, trigger, and echo pins.
  - Use a breadboard if necessary to organize and connect the circuit components.
- ***Programming (30 minutes):***
  - Write the Arduino code to control the robot's movements and process sensor data for obstacle detection.
  - Implement an algorithm for obstacle avoidance, such as a simple "move forward until obstacle detected, then turn" approach.
  - Test and debug the code to ensure proper functionality and adjust parameters as needed for optimal performance.
- ***Testing and Troubleshooting (20 minutes):***
  - Power up the robot and deploy it in a controlled environment with obstacles (e.g., cardboard obstacles, books).
  - Observe the robot's behavior as it navigates the environment, noting any issues with obstacle detection or movement.
  - Troubleshoot any problems encountered, such as adjusting sensor placement, fine-tuning motor control, or refining the obstacle avoidance algorithm.



- ***Data Analysis (10 minutes):***
  - Analyze the robot's performance during testing, including its ability to detect obstacles accurately and navigate around them.
  - Discuss any limitations or challenges encountered during the testing process and brainstorm potential improvements or modifications.
- ***Project Reflection (10 minutes):***
  - Reflect on the project as a group, discussing what was learned and achieved.
  - Share insights into the design and implementation process, highlighting successes and areas for improvement.
  - Discuss real-world applications of obstacle avoidance robots and how the project could be expanded or adapted for different scenarios.

Note: This project provides an opportunity for hands-on learning in robotics, combining mechanical assembly, circuitry, programming, and problem-solving skills to create a functional obstacle avoidance robot. Encourage creativity and experimentation throughout the project to foster innovation and deeper understanding.



ARTIFICIAL INTELLIGENCE

Pre-Activity Preparation

- Team Formation: Form teams ensuring a mix of abilities and interests. Consider an odd number of members in each team to avoid ties during decision-making.
- Schedule Planning: Allocate time for each activity, including setup and wrap-up. Ensure a balance between active and reflective phases.

Activity 1

The Turing Test Debate

Objective:

Encourage critical thinking and debate on the Turing Test, its implications, and challenges in assessing machine intelligence.

Material Needed

Whiteboard/chalkboard, markers/chalk, index cards/sticky notes, timer/stopwatch.

Procedure:

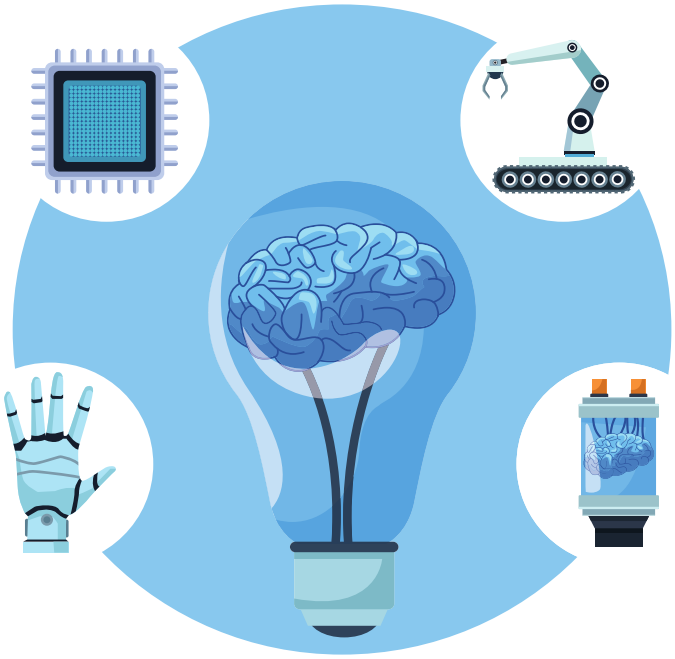
- ***Introduction (10 mins)***:
  - Explain the Turing Test's significance in AI history, measuring a machine's ability to exhibit indistinguishable intelligent behavior from humans.
- ***Activity Setup (5 mins)*:**
  - Divide the class into Interrogators (Group A) and Machines (Group B). Assign roles and fictional identities to Machines.
- ***Turing Test Simulation (20 mins):***
  - Machines aim to convince Interrogators they are human.
  - Interrogators engage in text-based conversations, guessing if they are talking to a human or machine.
  - Use timer for each conversation (3-5 mins).
- ***Discussion and Reflection (15 mins):***
  - Facilitate discussion on Machines' strategies, moments of doubt, and challenges in determining machine intelligence from conversation alone.
- ***Debrief (10 mins):***
  - Discuss results, revealing which students were Machines and their fictional identities.
  - Explore limitations and criticisms of the Turing Test. Ask students if it's a valid measure of machine intelligence.
- ***Conclusion (5 mins):***
  - Summarize key takeaways, emphasizing the Turing Test's thought-provoking nature and its impact on debates about intelligence and AI. Encourage reflection on ethical and philosophical implications.

This activity promotes engagement, critical thought, and ethical considerations in the context of AI and the Turing Test.



Assignments:

- Implement simple programs in Python using modules and packages like NumPy, Pandas, SciPy, and Matplotlib.
- Create and manipulate lists in Python, demonstrating operations like append, insert, extend, sort, and search.
- Explore the characteristics of tuples, including creation, access, deletion, and conversion from lists to tuples.



Activity 2

Cybersecurity Challenge: Protect the Digital Fort

Objective:

To raise awareness about cybersecurity best practices and the potential consequences of security breaches.

Material Needed

- Whiteboard or chalkboard
- Markers or chalk
- Index cards or sticky notes
- Access to a computer with internet access
- A list of cybersecurity scenarios or challenges

Procedure:

- ***Introduction (10 minutes)*:** Start by explaining the importance of cybersecurity in our digital world. Emphasize the role of individuals in safeguarding sensitive information and digital assets.
- ***Scenario Setup (10 minutes)*:**
  - Create a list of cybersecurity scenarios or challenges that reflect common threats and vulnerabilities. These can include scenarios like phishing emails, password breaches, malware infections, and unauthorized access attempts.
  - Write each scenario on an index card or sticky note, ensuring that they are concise and understandable.
- ***Scenario Presentation (15 minutes)*:**
  - Gather the students and present one cybersecurity scenario at a time. Read or display the scenario on the whiteboard.
  - Ask the students to brainstorm and discuss possible actions they would take to protect themselves or their organization in that situation. Encourage them to think about preventive measures and responses.
  - As a group, compile a list of cybersecurity best practices for each scenario on the board.
- ***Cybersecurity Challenge (20 minutes)*:**
  - Divide the class into small teams or pairs.
  - Distribute a set of index cards or sticky notes, each containing one of the cybersecurity scenarios you prepared earlier, to each team.
  - Instruct each team to discuss and write down a plan of action for their assigned scenario. They should consider both preventive measures and responses to the threat.
- ***Simulation (15 minutes):***
  - Have each team present their scenario and cybersecurity plan to the class.
  - Encourage discussion and debate about the effectiveness of the proposed solutions.
  - After each presentation, reveal the best practices and responses, emphasizing key cybersecurity principles.
- ***Discussion (10 minutes):***
  - Facilitate a class discussion about the importance of cybersecurity, the challenges individuals and organizations face, and the potential consequences of not following best practices.
  - Discuss the real-world implications of cybersecurity breaches, including data theft, financial losses, and reputation damage.
- ***Conclusion (5 minutes)*:** Summarize the key takeaways, highlighting the importance of proactive cybersecurity practices in protecting digital assets and personal information.



Activity 3

Automated or Autonomous

Objective:

To help students differentiate between automated and autonomous systems and understand their practical applications.

Material Needed

- Whiteboard or chalkboard.
- Markers or chalk.
- Sticky notes or index cards.
- Examples of automated and autonomous systems (e.g., a programmable thermostat for automated and a self-driving car for autonomous).

Procedure:

- ***Introduction (5 minutes):*** Begin by explaining the difference between automated and autonomous systems. Emphasize the key points:
  - **Automated Systems:** Follow predefined rules and instructions, predictable outcomes.
  - **Autonomous Systems:** Adapt and make decisions in real-time, handle uncertainty.
- ***Interactive Activity (15 minutes):***
  - Divide the whiteboard into two sections, one labeled "Automated" and the other labeled "Autonomous."
  - Provide students with a list of scenarios or examples of systems and ask them to decide whether each belongs in the "Automated" or "Autonomous" section. You can use sticky notes or index cards for this.
  - Examples for consideration might include:
    - A washing machine
    - A self-checkout kiosk at a store
    - A self-driving car
    - A coffee maker
    - A robotic vacuum cleaner
    - A GPS navigation system
- ***Discussion (10 minutes):***
  - Go through each scenario or example with the students, asking them to explain their reasoning for categorizing it as either automated or autonomous.
  - Discuss the gray areas where the distinction may not be entirely clear and explore real-world examples where automated and autonomous systems coexist (e.g., automated assembly lines in car manufacturing with autonomous robots for certain tasks).
- ***Wrap-up (5 minutes):*** Summarize the key takeaways from the activity, highlighting the importance of understanding the differences between automated and autonomous systems, especially as AI technology continues to evolve.



Activity 4

Human vs. Machine Decision Challenge

Objective:
To illustrate the differences between human and machine decision-making processes through a fun and interactive challenge.

Material Needed

- Whiteboard or chalkboard
- Markers or chalk
- Index cards or sticky notes
- Two groups of students (Group A representing humans and Group B representing machines)

Procedure:

- ***Introduction (5 minutes):*** Begin by explaining the concept of decision making and the differences between human and machine decision-making processes, as discussed in your primer.
- ***Interactive Activity (20 minutes):***
  - Divide the class into two groups: Group A (humans) and Group B (machines). If the class is small, you can assign roles individually; otherwise, assign roles to small teams within each group.
  - Provide each group with a set of index cards or sticky notes. These will serve as "decision cards."
  - Present a series of decision-making scenarios or problems to both groups. These scenarios can be hypothetical situations or real-world examples from various domains (e.g., medical diagnosis, image recognition, customer service).
  - For each scenario, both groups must make a decision by writing it down on their decision cards. Group A (humans) should make decisions based on their judgment and intuition, while Group B (machines) should make decisions based on predefined rules or algorithms (you can provide these rules in advance).
  - Discuss the decisions made by both groups for each scenario. Compare and contrast the thought processes, reasoning, and outcomes of human and machine decision-making.
  - Encourage discussion and debate among students. Ask questions like:
    - Were the decisions of Group A (humans) influenced by emotions or personal beliefs?
    - Did Group B (machines) consistently follow the predefined rules, or were there limitations in their decision-making?
- ***Debrief (10 minutes):***
  - Summarize the key takeaways from the activity, emphasizing the strengths and weaknesses of both human and machine decision-making.
  - Discuss the importance of combining the strengths of both humans (subjective judgment) and machines (data-driven analysis) in real-world applications.
- ***Reflection (5 minutes):*** Ask students to reflect on how this activity has changed their perception of decision- making and the role of AI in it.



Activity 5

Teaching a Machine to Sort Fruits

Objective:

To introduce students to the concept of machine learning by using a simple and relatable example of teaching a machine to sort fruits based on their characteristics.

Material Needed

- Various plastic or toy fruits (e.g., apples, bananas, oranges)
- A computer or tablet with internet access
- Access to a simple machine learning platform or tool (e.g., Google's Teachable Machine)

Procedure:

- ***Introduction (10 minutes):*** Start by explaining the concept of machine learning to the students, emphasizing that it's a way to teach computers to make decisions or predictions based on data.
- ***Activity Setup (10 minutes):***
  - Arrange the various plastic or toy fruits in front of the class.
  - Explain that the task is to teach a virtual machine (computer) to recognize and sort these fruits automatically based on their appearance.
- ***Teachable Machine (15 minutes):***
  - Use a computer or tablet to access a simple machine learning platform like Google's Teachable Machine (teachablemachine.withgoogle.com).
  - Briefly demonstrate how to use the platform to train a machine learning model. You can use this tool to train a model to recognize and sort the toy fruits.
  - Show students how to capture images of each type of fruit and assign labels (e.g., "apple," "banana" "orange").
- ***Student Participation (20 minutes):***
  - Divide students into small teams or pairs.
  - Provide each team with access to the Teachable Machine platform and a designated fruit.
  - Instruct each team to train the machine learning model to recognize their assigned fruit. They should capture multiple images of the fruit from different angles.
  - Encourage students to experiment and adjust their training until they achieve satisfactory recognition.
- ***Testing and Competition (15 minutes):***
  - After training, have each team test their machine learning model by placing the assigned fruit in front of the computer's camera.
  - Keep score based on how accurately each machine recognizes and sorts the fruits.
  - Encourage friendly competition among teams to see which one achieves the highest accuracy.
- ***Discussion (10 minutes):***
  - Gather the class to discuss the activity. Ask students questions like:
    - How did the machine "learn" to recognize fruits?
    - What challenges did you face in training the model?
    - What other real-world applications can you think of for this kind of machine learning?
- ***Conclusion (5 minutes):*** Summarize the key takeaways from the activity, emphasizing that machine learning involves training computers to make predictions based on patterns in data. Discuss the broader applications of machine learning in various fields.



VI. Some General Activities for Robotics and AI Club

Robotics Quiz

Quiz can be conducted, and the level can be kept as beginner.
Here are some fact-based multiple-choice questions for a beginner-level robotics quiz, you can always add more.

1. Which of the following is a basic component of a robot?
   (i) Operating System
   (ii) Sensor
   (iii) Music Player
   (iv) Smartphone
   Answer: (ii) Sensor

2. What is the primary function of a proximity sensor in a robot?
   (i) Measure temperature
   (ii) Detect nearby objects
   (iii) Play music
   (iv) Calculate distances
   Answer: (ii) Detect nearby objects

3. Which programming language is commonly used for programming robots in educational settings?
   (i) Java
   (ii) Ruby
   (iii) Python
   (iv) C++
   Answer: (iii) Python

4. What does "AI" stand for in the context of robotics?
   (i) Always Innovative
   (ii) Advanced Intelligence
   (iii) Artificial Intelligence
   (iv) Actual Interaction
   Answer: (iii) Artificial Intelligence

5. Which of the following laws were proposed by Isaac Asimov for the behavior of robots?
   (i) Newton's Laws
   (ii) Kepler's Laws
   (iii) The Laws of Robotics
   (iv) Ohm's Laws
   Answer: (iii) The Laws of Robotics

These multiple-choice questions focus on essential facts and concepts in robotics and are suitable for a beginner-level quiz.



Robotics Project

Activity

Remote-Controlled Surveillance Robot

Objective:

Build a remote-controlled surveillance robot for enhanced security.

SDG 16 - Peace, Justice, and Strong Institutions

NSQF Standards:

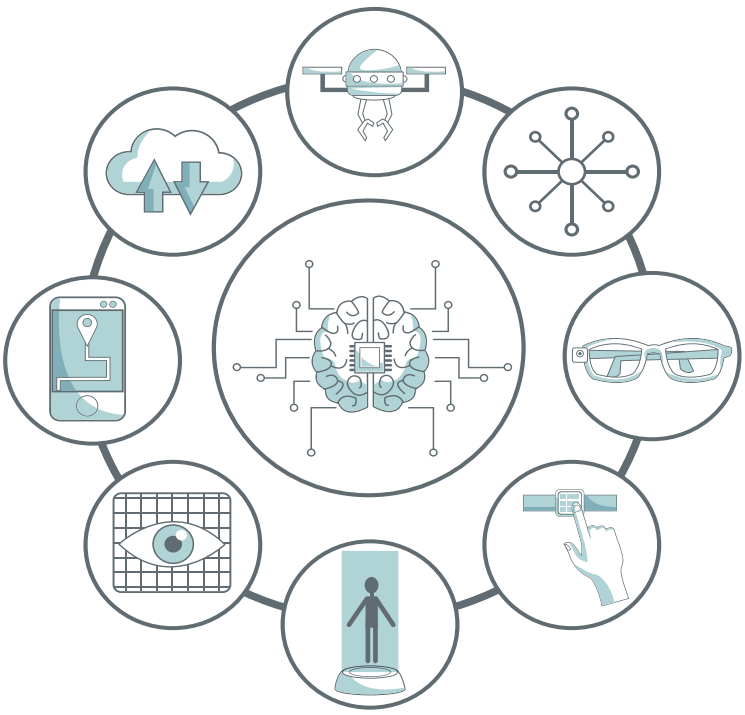
Level 4 (Expert Project Development)

Material Needed

- Robot chassis
- Raspberry Pi or Arduino
- Webcam or camera module
- Motor driver
- Wheels and motors

Procedure:

- Assemble the robot chassis and attach the camera.
- Program remote control capabilities for surveillance tasks.
- Improve security measures and reduce human risk.



Artificial Intelligence Project

Activity

Climate Change Prediction Using AI

Objective:

Develop an AI-based project to analyze climate data and predict future climate changes for climate action and adaptation.

SDG 9: Industry, Innovation, and Infrastructure

SDG 13: Climate Action

NSQF Standards:

Level 4 (Expert Project Development)

Material Needed

- Climate data sets
- AI programming tools (e.g., Python, TensorFlow)
- Machine learning models
- Computing resources

Procedure:

- Collect and preprocess climate data.
- Select key features and relevant variables.
- Choose appropriate machine learning models.
- Train models, validate, and test for accuracy.
- Implement models for predicting future climate changes.
- Visualize predictions through graphs and maps.
- Document the entire process, including limitations.



VII. Sample Lesson Plan

Robotics

Lesson Plan:
Visualization, Design, and Creation of Components in Robotics

Objective:

Students will understand the application of mechanical blocks in robotics.
Students will be able to visualize, design, and create components of a robot using Tinkercad.
Students will learn about different types of joints, specifically revolute and prismatic, and the RR mechanism.
Students will be able to use Tinkercad to visualize the motion of the designed components.

Lesson 1: Introduction to Robotics and Mechanical Blocks	
Theory	Practical
Overview of robotics and its applications. Introduction to mechanical blocks in robotics.	Demonstration of basic robot components using Tinkercad. Hands-on activity: Designing a simple robot structure using Tinkercad.

Lesson 2: Types of Joints in Robotics	
Theory	Practical
Explanation of different types of joints in robotics (revolute and prismatic). Understanding the purpose and application of each type of joint.	Designing and creating models of robots with revolute joints in Tinkercad. Designing and creating models of robots with prismatic joints in Tinkercad.

Lesson 3: RR Mechanism	
Theory	Practical
Introduction to the RR mechanism and its significance. Understanding how RR mechanism contributes to robot motion.	Building a simple RR mechanism using Tinkercad. Experimenting with different configurations of the RR mechanism.



Lesson 4: Visualizing Motion in Tinkercad	
Theory	Practical
Overview of motion visualization in robotics. Introduction to Tinkercad's motion simulation features.	Applying motion to the previously designed robot components. Observing and analyzing the simulated motion in Tinkercad.

Lesson 5: Integration and Troubleshooting	
Theory	Practical
Understanding the importance of integration in robotics. Troubleshooting common issues in robot design.	Integrating the designed components into a complete robot model. Identifying and solving any issues that arise during integration.

Lesson 6: Project Work and Presentations
Practical
Students work on individual or group projects to design and simulate a more complex robot. Presentations of the projects, highlighting design choices, motion simulation results, and any challenges faced.

Assessment:

Continuous assessment during practical sessions.

Evaluation of the final project and presentation.

Quiz on theoretical concepts.

Resources:

Tinkercad tutorials and documentation.

Robotics textbooks and online resources.

Peer collaboration for project work.

Note: Adapt the lesson plan based on the duration of each session, the proficiency of the students, and the availability of resources. Encourage creativity and critical thinking throughout the practical sessions.



Artificial Intelligence

Lesson Plan:
Introduction to Data and Programming with Python

Objective:
Students will understand the concept of modules, packages, and libraries in Python.
Students will be able to use essential libraries like NumPy, Pandas, SciPy, and Matplotlib.
Students will learn about Lists, Tuples, and Strings in Python, along with their operations and methods.
Students will gain practical experience through hands-on coding exercises.

Lesson 1: Introduction to Modules and Packages

Theory	Practical
Understanding the need for modules and packages.	Writing simple programs to import and use NumPy for numerical operations.
Overview of libraries like NumPy, Pandas, SciPy, and Matplotlib.	Using Pandas for basic data manipulation and analysis.
Syntax for importing libraries in Python.	Plotting simple graphs with Matplotlib.

Lesson 2: Lists and Tuples in Python

Theory	Practical
Introduction to Lists: Definition, characteristics, and uses.	Coding exercises to create, manipulate, and perform operations on lists.
Creating and initializing lists.	Hands-on practice with tuple creation, access, and conversion.
Accessing elements of a list and list operations (append, insert, extend, sort, search).	
Introduction to Tuples: Definition, characteristics, and applications.	
Creating, accessing, and deleting tuples.	
Converting lists to tuples and vice versa.	

Lesson 3: Strings in Python	
Theory	Practical
Introduction to strings: Definition and properties. String operations: concatenation, escape sequences. String operators and in-built functions: len(), upper(), lower() endswith(), startswith() find(), islower() replace(), count() isalpha(), isdigit(), isalnum() join(), slicing [start:end:step].	Writing code to perform string operations. Implementing various string functions in practical examples.

Lesson 4: Comprehensive Practical Session
Practical
Integrating concepts from previous lessons into comprehensive coding exercises. Solving real-world problems using NumPy, Pandas, and Matplotlib. Manipulating data structures (lists, tuples, and strings) in practical scenarios.

Assessment:

Continuous assessment during practical sessions.

Coding assignments and projects.

Quiz on theoretical concepts and syntax.

Resources:

Python development environment (IDE or online compiler).

Online documentation for NumPy, Pandas, and Matplotlib.

Sample datasets for data manipulation and analysis.

Note: Adjust the pace of the lessons based on the students' understanding and the available class time. Encourage active participation and problem-solving during practical sessions.



Good to Know:

VIII. Future Pathways in Robotics and AI

A. Employment Prospects:

- As students progress in their education after the 9th and 10th grades, they can anticipate a growing demand for skills in Robotics and AI across various industries. Job prospects include roles such as:

***Robotics Technicians*:** Involved in the assembly, maintenance, and troubleshooting of robotic systems.

***AI Assistants Developers*:** Focusing on creating AI-driven virtual assistants for various applications.

***Programming Apprentices*:** Learning coding skills to program and control robots and AI systems. The students can gain skills like, Data Analysis, Data Engineering, Front-end developer, back-end developer, etc.

***Automation Support Roles*:** Engaging in tasks related to automated processes in industries.

B. Entrepreneurship Opportunities:

For those inclined towards entrepreneurship, there are numerous opportunities in the realm of Robotics and AI. Students can explore:

***AI Tutoring Services*:** Offering personalized AI-driven tutoring services for students.

***Robotics Kits for Education*:** Developing and selling kits that allow students to build and program their robots.

***AI-Enhanced Educational Apps*:** Creating apps that utilize AI for personalized learning experiences.

***Automation Solutions for Local Businesses*:** Providing automation solutions to local businesses to enhance efficiency.



C. Higher Education Pathways:

Preparation for higher education in Robotics and AI can begin with selecting the right subjects and courses:

***Mathematics and Computer Science*:** Build a strong foundation in mathematics and coding, essential for advanced studies in Robotics and AI.

***Physics and Electronics**:* Understanding the principles behind robotic systems and AI technologies.

***Introduction to Programming Languages*:** Learn languages like Python and Java, crucial for AI and robotics programming.

Online Courses and Certifications: Enroll in online courses and certifications offered by platforms like Coursera, edX, or Khan Academy to gain additional skills.

***Participation in Robotics Competitions*:** Joining robotics competitions at school or community levels to apply theoretical knowledge in practical scenarios.

***Exploration of STEM (Science, Technology, Engineering, and Mathematics) Programs*:** Seek out STEM programs that provide hands-on experiences in Robotics and AI.

By focusing on these pathways, students can lay the foundation for successful careers, entrepreneurial ventures, and higher education in the dynamic fields of Robotics and AI.



Good to Know:

IX. Inter-disciplinary connections of Robotics and AI

The illustration depicts the alignment between the robotics and AI course and fundamental subjects like Science, Mathematics, Biology, Economics and more.

Science Exploration of different types of robots (industrial, humanoid, drones) Presentation on discussing the impact of robotics on society Case Study on real-world examples of robots in different sectors	Mathematics Applying geometric concepts to robot motion and navigation Simulation exercises using geometry in robot programming. Designing paths and trajectories for robots
Economics and Social Science Case studies on companies adopting robotics and their economic outcomes. Group project: Prepare a cost benefit analysis report for a hypothetical industry. Presentation on cultural and societal differences in embracing robotics.	Biology Hands-on-activity: Design a robot inspired by a biological organism. Case studies on surgical robots and their biological counterparts. Presentation on finding the intersection of robotics and biology research.

Robotics

Figure 39: Academic subjects potentially aligned with robotics.

English: Explore AI tools for better understanding and interpreting literary works that involve themes related to light. Encourage students to use AI powered language tools for creative writing related to the beauty and symbolism of light.	Physics: Discover how AI powered simulations help in understanding the behaviour of light in everyday scenarios, like refraction and reflection. Explore interactive simulations that make learning optics concepts engaging.
Biology: Understand how AI is used in medical imaging, like X-rays, which involves the principles of light. Explore AI applications in analusing medical images to identify patterns and diagnose conditions.	Geography: Understand how AI is used in remote sensing technologies, like satellite imagery, which captures information using light. Eplore how AI aids in analysing geographical data collected through light-based sensors.

AI

Figure 40: Academic subjects potentially aligned with AI.





COUNCIL FOR THE INDIAN SCHOOL CERTIFICATE EXAMINATIONS